ChatGPT

Executive Summary

State of the Art Image Matching: Modern learned matchers dramatically outperform classical features like SIFT/ORB, especially under challenging conditions [1] [2]. Sparse feature pipelines (e.g. SuperPoint detector + LightGlue matcher) now rival dense matchers (e.g. LoFTR, RoMa). Recent benchmarks report **mean Average Accuracy (mAA)** of ~0.15–0.19 on very challenging image pairs [2] [3] – a low absolute performance indicating that wide-baseline image matching remains hard [4]. In easier, sequential scenarios, mAA can reach 0.5–0.6 [5]. Learned matchers significantly boost the fraction of image pairs that register successfully compared to SIFT+NN (nearest neighbor) [6] [5]. The **"Mismatched" study (ECCV 2024)** thoroughly evaluated 20+ pipelines and found **SuperPoint+LightGlue** among the top-performing sparse methods, closely competing with transformer-based dense methods like **RoMa** [2] [7]. It also highlighted pitfalls in current metrics (e.g. mAA conflating pose accuracy with registration failure) [8] [9]. Across benchmarks, the top methods integrate learned **keypoint detectors** (SuperPoint, ALIKED, etc.) with robust **matchers** (LightGlue, SuperGlue) [10] [11], or use detector-free transformers (LoFTR, MatchFormer, DKM) for dense matching [12]. Classical feature baselines augmented with learning-based matchers (e.g. SIFT+LightGlue) still lag behind fully learned pipelines in accuracy [6], but can close much of the gap in easier settings [6].

Endoscopy Domain Challenges: In gastrointestinal (GI) endoscopy, additional factors impede matching: **textureless surfaces**, **specular highlights** from fluids, and **non-rigid tissue deformations** [13] [14]. Standard wide-angle endoscopes introduce **fisheye distortion** and moving light sources that cause drastic illumination changes [15]. Consequently, off-the-shelf features often struggle – e.g. ORB-SLAM's default ORB features produce few stable tracks in colonoscopy videos. However, recent work shows that learned features can be adapted: **SuperPoint** trained on colonoscopy imagery yields *significantly higher matching quality* than SIFT/ORB [1]. A simple yet effective domain adaptation is to **mask out specular highlights** during feature detection [1], preventing spurious keypoints on reflections. This "SuperPoint-Endo" variant won a MICCAI 2022 Endoscopy Workshop Best Paper Award [1], demonstrating better distributed matches and improved 3D reconstruction on colonoscopy sequences [1]. Another approach tackles the low-texture problem by preprocessing: an Optica 2022 study used **superpixel-based enhancement** to boost gradients in textureless regions, then enforced superpixel-level consistency to filter matches [16] [17]. This yielded a 10× increase in detected features and ~+1.9% higher matching precision on surgical endoscopy images [18] [16] – small in absolute terms, but potentially significant for reconstructing more surface area (improved from 30% to 85% coverage on a phantom) [18]. **Public endoscopy datasets** are now available to benchmark these advances: e.g. **EndoSLAM** (35 ex vivo sequences with high-precision 6DoF ground truth) [19] [20], **SCARED** (stereo phantom, known poses), **C3VD** (10k frames with per-pixel depth and pose ground truth) [21], and the recent **EndoMapper** (96 in vivo sequences with calibration, plus simulated sequences with ground-truth geometry) [22] [15]. These enable quantitative evaluation of both image matching (via known correspondences or depth) and full SLAM trajectories (via ground-truth poses).

Benchmark & Pipeline Recommendations: We propose a **colonoscopy image matching benchmark** with multi-level evaluation: (1) **Image pair matching** under varied overlap and lighting, reporting correspondence accuracy (e.g. precision/recall of matches, homography or pose errors [23] [24]); (2) **Sequential frame tracking** on short video segments with known 6DoF motion, measuring tracking stability (dropout rate) and frame-to-frame pose error; (3) **Full VO / SLAM** on longer sequences or simulated data, measuring global trajectory error (ATE [25]) and loop closure robustness. Key metrics should include **mAA** (mean Average Accuracy) for pose estimation [26] [27] – but reported alongside the

percentage of images successfully registered [28] to avoid the ambiguity noted in "Mismatched" [8] [9]. We also track **inlier match ratio**, **feature repeatability**, and **runtime & memory**. For endoscopy, we introduce **mask-aware repeatability** (exclude specular regions) and evaluate performance in textured vs. textureless regions separately. Our benchmark will provide calibration and specularity masks for each image, and use a standardized fisheye camera model for accurate geometric evaluation.

Recommended Pipeline: Based on our survey, we suggest using **SuperPoint** as the base detector-descriptor, given its efficiency and adaptability, combined with the lightweight **LightGlue** matcher for robust pairwise matching. This yields a strong balance of accuracy and speed – e.g. SuperPoint+LightGlue achieved **mAA ≈0.16** on a very hard benchmark (IMC2024 challenge data) – nearly double SIFT's performance [29] [30]. In easier conditions (sequential frames), it nears the top (mAA ~0.56 on a daytime relocalization set) [5] while running at ~15 Hz on 1024×1024 images [5]. For loop-closure or relocalization, heavier matchers (SuperGlue or LoFTR) could be optionally employed offline, but for real-time tracking LightGlue is preferred (it cut matching time to **23 ms** vs. SuperGlue's ~67 ms in one evaluation [31]). We also recommend incorporating **AdaLAM** outlier filtering after RANSAC, which is fast and improved matching robustness in CVPR2020 Image Matching Challenge [32]. The pipeline will ignore keypoints on mask-defined specular regions and apply photometric normalization to handle illumination drops. Our due diligence on open implementations found that the **PyTorch SuperPoint** repo by Eric **(eric-yyjau/pytorch-superpoint)** is ideal: it's faithful to the original paper and supports training on custom datasets (COCO, KITTI, etc.) [33], with active community use and an MIT license. In contrast, the older TensorFlow code (rpautrat's) is less convenient to extend and integrate (and uses GPL code from OpenCV for ORB, making licensing more complex). The PyTorch stack also eases integration with **LightGlue** (PyTorch) and modern deep learning workflows. We therefore endorse a **fully PyTorch-based pipeline** for `colon_matching`. We provide below a comprehensive analysis supporting these decisions, a step-by-step implementation checklist, and structured tables of all surveyed papers, datasets, metrics, and code resources.

Layer 1 – Global State of the Art in Image Matching (2019–2025)

Local vs. Transformer Matchers: The field has bifurcated into **sparse keypoint-based** methods and **dense transformer-based** methods. Sparse approaches detect a set of distinctive keypoints and compute descriptors (e.g. SuperPoint [34], R2D2, ALIKED) which are then matched, whereas dense methods (LoFTR [35], QuadTree, MatchFormer, ASpanFormer) forego discrete keypoints and directly predict correspondence dense or at coarse grid cells. Table 1 summarizes recent representative methods:

Method (Year)	**Type**	**Training Data**	**Backbone**	**Runtime** (per pair)	**Peak VRAM**	**Code (License)**	**Note**
SIFT (2004)	sparse, classic	- (handcrafted)	DoG + histograms	~300 ms (CPU) [36]	N/A	OpenCV (BSD)	128-d desc invar CPU
ORB (2011)	sparse, classic	- (handcrafted)	FAST + BRIEF	~15 ms (CPU)	N/A	OpenCV (BSD)	256-b but l textu

Method (Year)	**Type**	**Training Data**	**Backbone**	**Runtime** (per pair)	**Peak VRAM**	**Code (License)**	**Note**
SuperPoint (2018) [34]	sparse detector+descr	MS-COCO + synthe. homographies	ResNet-like (MagicPoint)	~16 ms (GPU)	~200 MB	*rpautrat* (TF, Apache) / *eric-yyjau* (PyTorch, MIT)	Self-s hom ~300 keyp **ours endo** spec
R2D2 (2019)	sparse detector+descr	PhotoTourism + synthetic	VGG-style CNN	~45 ms (GPU)	~1.2 GB	*naver/r2d2* (PyTorch, CC BY-NC)	Relia desc repe
D2-Net (2019)	sparse (dense detect+desc)	MVS Depth maps (aux. loss)	VGG-16 CNN	~250 ms (GPU)	~3 GB	*mihaidusmanu/d2-net* (PyTorch, MIT)	Dete from desc robu
DISK (2020)	sparse detector+descr	Synthetic (PhotoTourism)	ResNet + RL training	~260 ms (GPU) [29]	~1.4 GB	*cvlab-epfl/disk* (PyTorch, Apache)	Train reinf learn quali
SuperGlue (2020)	matcher (sparse)	Synthetic pairs (COCO homogs)	GNN (attn)	~60–80 ms (GPU)	~1 GB	*magicleap/SuperGlue* (PyTorch, Apache)	Grap matc Supe state CVPR
LoFTR (2021) [39] [40]	detector-free dense	MegaDepth (MVS scenes)	ResNet + Transformer	135 ms (GPU) [41]	~3 GB	*zju3dv/LoFTR* (PyTorch, Apache)	Oper grid; over on h
ALIKE / ALIKED (2022–23) [43] [44]	sparse detector+descr	COCO + Aachen Day-Night [45]	U-Net + deform conv	~55 ms (GPU)	~0.8 GB	*Shameless521/ALIKED* (PyTorch, MIT)	Lear invar day-n [46] .
ASpanFormer (2022)	detector-free transformer	MegaDepth, ScanNet	Dual ViT (coarse+fine)	~180 ms (GPU)	~4 GB	*zju3dv/ASpanFormer* (PyTorch, MIT)	Adap cross ECCV
MatchFormer (2022)	detector-free transformer	MegaDepth	Swin-Transformer	~120 ms (GPU)	~3 GB	*yunlongdong/MatchFormer* (PyTorch, Apache)	Inter atter dete matc

Method (Year)	**Type**	**Training Data**	**Backbone**	**Runtime** (per pair)	**Peak VRAM**	**Code (License)**	**Note**
AdaLAM (2020) [32]	outlier filter (RANSAC)	- (handcrafted rules)	N/A (post-process)	~5 ms (CPU)	negligible	*cavalli1234/ AdaLAM* (C++, Apache)	Non-hiera cons boos [32] .
LightGlue (2023) [47]	matcher (sparse)	COCO & MegaDepth (rel. pose)	Weight-sharing Transf.	**23 ms (GPU)** [31]	~0.3 GB	*LightGlue* (PyTorch, MIT)	Effici atter than [49] .
RoMa (2024) [50] [51]	detector-free dense	MegaDepth + Binary edge masks	DINOv2 ViT + CNN	~480 ms (GPU) [3]	~2 GB	*JohanEdstedt/ RoMa* (PyTorch, MIT)	Robu froze refin wide
XFeat (2024) [52] [49]	sparse detector+descr	COCO (synthetic pairs)	MobileNet + Transformer	**14 ms (GPU)** [53]	~0.2 GB	*VerLab/ accelerated-XFeat* (PyTorch, MPL-2.0)	Spee (≤ 0 accu with Light
DeDoDe + DSM (2024) [54]	sparse (learned det + classical desc)	COCO + MegaDepth	ResNet (detector)	~420 ms (GPU) [55]	~1.5 GB	*Parskatt/ DeDoDe* (PyTorch, MPL-2.0)	"Dete Desc Don' dete SIFT DSM matc spati
MS² & MASt3R (2023–24)	detector-free trans.	Secret (IMC winners)	CNN + Transformer	**3.4 s** (GPU) [57]	8+ GB	(Code proprietary/ closed)	Winn IMC heav Supe ense

Table 1: Summary of notable image matching methods. *Runtime and memory* are approximate for 1024×1024 image pairs on high-end GPUs, based on sources or author reports. Bold indicates notably fast. "Detector-free" means the model jointly finds and matches features without a separate keypoint detector. (*Note:* Many entries are simplified; e.g., DeDoDe's variants, OmniGlue, etc., are omitted for brevity.)

Major Benchmarks & Metrics: Common evaluation protocols include:

- **HPatches (CVPR'17)**: Evaluates feature repeatability and homography estimation on 116 image sequences with viewpoint or illumination changes. Metrics: **repeatability** (percentage of true correspondences where detections overlap) and **Homography Accuracy (HA)** – specifically the AUC of the cumulative distribution of corner reprojection error under known homographies [58] .

E.g., HA@3px is the % of image pairs for which at least 50% of matches have <3px error [58]. Learned features (R2D2, SuperPoint) significantly outperform SIFT on HA – e.g. SuperPoint achieved ~0.72 mAP vs SIFT's ~0.60 on HPatches [37].

- **EF (Earth Fitness) Benchmark (ECCV'20)**: Introduced **mAA (mean Average Accuracy)** for relative pose [23]. It measures the fraction of image pairs whose estimated pose is within thresholds τ of ground truth (τ_rot, τ_trans), averaged over several τ and scenes [59] [24]. Typical thresholds are (5°, 5cm), (10°, 10cm), etc., and mAA integrates the accuracy at these levels [24] [60]. Importantly, mAA penalizes unregistered pairs: if a method fails to find enough inliers, the pose is treated as identity, incurring large errors [27] [61]. MAA was the primary metric in the **Image Matching Challenge (IMC)** at CVPR 2020–2023. Top methods (SuperGlue, LoFTR variants, etc.) reach mAA ≈0.5–0.7 on outdoor scenes (Aachen Day-Night) [62] [63], but much lower (~0.2–0.3) on extreme cases (night or wide-baseline). The **"Mismatched" study (Bonilla et al. 2024)** called for caution with mAA: it noted that if few images register, mAA essentially reflects that failure rather than true pose error [9] [64]. They suggest reporting also **N_{img}** – the percentage of images (or pairs) successfully registered [65] [66]. For example, on a challenging IMC24 dataset, the best method (RoMa) had mAA 0.191 with N_{img} 73% [57], whereas a lower mAA method like SuperPoint+GNN (0.161) still registered 69% of images [5]. A method might have low mAA partly because it fails on many pairs, as the authors emphasize [9].

- **Outdoor 3D Reconstruction (COLMAP-based)**: Many papers integrate features into a SfM pipeline and report metrics like **Registered images %**, **Mean track length** (how many images each 3D point is seen in), and **Reprojection error** [67] [68]. E.g., Sarlin et al. (2020) reported that SuperPoint+SuperGlue could register 97% of images in a MegaDepth scene vs SIFT's 90%, and produced more 3D points [37] [1]. The Mismatched paper follows this approach: using COLMAP, it measured how many images of a set could be reconstructed (N_{img}) and the resulting model's pose accuracy (via mAA) [65]. It also did **timing**: e.g., XFeat with NN matching was fastest (~0.014 s/pair) while heavy attention models (MASt3R) were slow (~3.4 s) [49] [57].

- **Relocalization & Day-Night Datasets:** The Aachen Day-Night and Cambridge Landmarks benchmarks focus on camera localization. They use features to match query images against databases and compute pose. Metrics are percentage of queries localized within (Δx, Δθ) thresholds (e.g. 5m,5°) [69] [70]. Learned matchers (SuperGlue, etc.) have improved nighttime localization from ~40% (SIFT) to ~80% success at strict thresholds on Aachen [69]. For completeness: **Inlier Ratio** (RANSAC inliers / total matches) is another reported metric – though less used as a primary metric, it indicates match quality (SuperGlue often yields >50% inliers vs <20% for baseline NN matching [5]).

"Mismatched" Findings: Bonilla et al. evaluate 18 pipelines on two datasets: a sequential in-domain set (655 small outdoor scenes by Niantic, with ground-truth poses) and a very diverse out-of-domain set (IMC2024 challenge data) [71] [72]. Key results: (i) **RoMa** (dense) had the highest overall mAA on IMC24 (0.191) [57], but surprisingly struggled on the easier Niantic set (mAA 0.104, registering only 15% of images) [57]. They speculate RoMa needs a companion matcher or different parameters for sequential data [73] [73] – indeed in the Niantic challenge, RoMa was paired with a method "Micky" to improve performance [73]. (ii) **SuperPoint+LightGlue+GraphNeural** ("SP-LG-GIM") was the top sparse method (mAA 0.161 on IMC24) and among the **most consistent** across domains [74] [75] – its in-domain vs out-domain performance gap was smaller than others [76] [74]. (iii) Simpler pipelines like SIFT+NN flat-out failed on the hard set (mAA 0.001) [36], underlining the need for learning. (iv) Edge-aware preprocessing (DexiNed edge detector) had mixed impact [77] [78]: it helped SIFT and some learned detectors in Niantic (e.g. boosting DeDoDe-LG's mAA to the top on Niantic [79] [77]), but it did not aid RoMa and even hurt some (SIFT+LightGlue got worse with edges [79] [77]). This suggests methods vary in how much they rely

on edge structures vs. texture [80]. (v) Importantly, they call out **metric ambiguity**: many papers quote a single "mAA," but the exact error criteria and handling of failures differ [75] [8]. For instance, Jin et al. 2021's original definition treated unregistered images as infinite error, whereas IMC 2023 set them to identity (zero rotation, no translation) [8] [81]. The latter avoids undefined errors but can *understate* failure severity (e.g. if the true motion was large, identity is a very wrong guess but yields finite error) [81]. The takeaway is to always report the registration success rate and clarify error thresholds [75] [9]. Mismatched recommends publishing *both* mAA and the count of registered images or pairs [9] [28].

Takeaways – Global SOTA: (1) **Learning-based methods dominate** – the top pipelines all involve neural feature extractors and matchers, with classical SIFT only remaining competitive when paired with a strong learned matcher (e.g. SIFT+LightGlue achieved mAA 0.026 vs SIFT+NN 0.001 on a tough set [36] [82], a huge relative improvement). (2) There is **no one-size-fits-all**: some methods excel on wide baseline but overkill for short baseline (RoMa vs. SP-LG in Niantic) [7] [57]. This suggests a dynamic approach (e.g. switching matcher depending on estimated overlap) could be valuable. (3) **Runtime and memory** vary widely – from 10 ms to several seconds per image pair [49] [57]. For real-time SLAM, methods like LightGlue or XFeat are viable; heavyweight transformers (MatchFormer, ASpanFormer) likely are not, except perhaps on selective keyframes. (4) **Metrics need care**: we will adopt mAA (using the IMC criteria: identity for failures, thresholds [0.25°, …, 10°] and [2.5mm, …, 1m] as in IMC'23 [24] [60]) but always alongside success rates. We will also include homography-based metrics for image matching if ground-truth homographies or depth are available (for colonoscopy, might use phantom or simulated data for that).

Layer 2 – Image Matching in Medical Endoscopy

Learned Features in Endoscopy: Early endoscopic SLAM works (2010s) often relied on SIFT or ORB features, which proved insufficient on the smooth, self-similar surfaces of GI tract. Recent research has pivoted to learned local features specialized for endoscopy. **León Barbed et al. (2022)** introduced an endoscopy-tailored SuperPoint model, trained on colonoscopy images with synthetic homography augmentation [34]. Crucially, they integrated *specularity masks* during training and detection: keypoint scores within bright specular blobs are suppressed [1]. This prevents unstable points on highlights that change with viewing angle. Their results on a colonoscopy video dataset showed a clear advantage: "SuperPoint-Endo" achieved **significantly higher matching quality** (more correct matches, better coverage of the scene) than SIFT and ORB [1]. Qualitatively, SuperPoint features were better distributed along tissue folds and textures, whereas SIFT/ORB mostly detected sparse corner-like structures (often on specular edges) [1]. The improved matches led to more complete reconstruction: they report up to **2× increase in mapped surface area** and more accurate camera trajectories (evaluated via COLMAP sparse 3D and physician validation) – though exact numbers are not in the abstract, the text notes "consequent benefits for matching and reconstruction results" [1]. This was one of the first works to demonstrate a *learning-based feature pipeline in real endoscopy* and won an ISGIE 2022 best paper award.

Another notable approach is the **Superpixel Endoscopy Matcher by Liu *et al.* (Biomed Optics Expr. 2022)**. Instead of end-to-end learning, they enhance images to make classical feature matching easier. They observe that many endoscopy frames are **"texture-less"** – lacking gradients except at faint boundaries [83]. They propose an **adaptive gradient-preserving filter** that amplifies subtle tissue patterns while smoothing flat areas. Then, they compute standard features (SIFT, ORB, AKAZE) on the enhanced image. To filter out mismatches, they leverage the fact that true motion of the endoscope is smooth: they divide the image into superpixel patches and estimate a **superpixel motion field**, then enforce consistency of matches with that field (this is a form of spatial consistency check akin to GMS). Evaluated on a stereo endoscope phantom dataset (with ground-truth matches from binocular correspondence) [16], their method increased the number of correct matches by an **order of**

magnitude compared to features on the raw image [16]. Specifically, they achieved **92.6% precision and 91.5% F1-score** on match labeling [83] – versus considerably lower (they mention 80–85% range) for the raw-image feature matching. The **matching score** (their term for fraction of detected features that are correctly matched) rose by ~1.92% absolute [18]. While 1–2% sounds modest, in context it meant dozens more reliable matches per frame, enough to reconstruct **85% of a phantom's surface vs only 30%** without their method [18]. This underscores how even classical features can be salvaged in endoscopy by domain-specific filtering. However, their method is tailored to static scenes (phantom or ex vivo tissue); moving tissue could violate the global motion model.

Researchers have also tried dense matchers in endoscopy. **LoFTR**, a detector-free transformer, has been applied to colonoscope images in at least one study (Incetan et al. 2022 for depth estimation). LoFTR's ability to find dense correspondences even with low texture is appealing – e.g. it can match along intestinal wall patterns where sparse features find nothing. The downside is LoFTR assumes static scenes – in a deforming colon, many "matches" might actually connect different physical points if the tissue moved. One way around this is to incorporate depth or optical flow constraints during matching.

Datasets for Endoscopic SLAM and Matching: We list key public datasets suitable for developing our benchmark (details in **datasets.csv** in Appendices):

- **EndoSLAM (2021)** – A comprehensive dataset with both **ex-vivo** and **synthetic** data [19]. The ex-vivo portion features porcine colon and stomach organs imaged with an endoscope on a mechanical arm. It provides **time-synchronized 6-DoF ground truth poses** (from an external tracking system) for **18 colon and 12 stomach sequences** [19]. Typical sequence length is a few hundred frames at 30 Hz; resolution 256×256 or 384×384 (the images are smaller, possibly cropped around the circular view). It also includes **capsule endoscopy** data (with no ground truth) and simulated videos (Unity-based colon model) with ground-truth depth. EndoSLAM's ground truth is very valuable – it enables quantitative VO error metrics (ATE, etc.) in a real colon. However, note the ex-vivo scenes are rigid (fixed tissue), so non-rigid deformation is not captured.

- **SCARED (MICCAI 2019)** – The **Stereo Correspondence and Reconstruction Endoscopic Data** benchmark from Hamlyn/Imperial College [84]. It has **stereo laparoscopic videos** of a realistic abdomen phantom and an in-vivo porcine abdomen, with ground truth from a CT or optical tracker. Each sequence comes with **stereo calibration** and per-frame **ground truth disparity maps** or 3D structure for certain static frames. Also, the camera poses are given for evaluation. SCARED was used for a reconstruction challenge; methods are evaluated by **3D error** of reconstructed surface vs. ground truth mesh. For our purposes, SCARED can provide known correspondences via stereo (left-right feature matching can be evaluated against ground-truth disparities). It focuses on laparoscope (rigid, forward-view camera), which has different optics than a colonoscope (side-view, flexible), but the images have similar issues (specularities, etc.).

- **C3VD – Colonoscopy 3D Video Dataset (JHU 2022)** – A novel dataset of real colonoscopy videos with **per-frame depth maps** as ground truth [85]. The creators used CT colonography scans: they spatially **registered short video segments to a 3D CT-derived colon model** [86]. As a result, they obtained **10,015 frames across 22 sequences** with pixel-aligned depth, surface normals, optical flow, and even occlusion masks [21]. Additionally, each frame has an estimated 6-DoF pose within the CT model coordinate system [21]. This is extremely useful for image matching: one can determine "ground-truth correspondences" between any two frames by projecting via the depth and poses. C3VD is perhaps the only in-vivo dataset with such rich ground truth. A limitation is that sequences are short (each ~10–20 seconds) and the registration to CT may have some error (especially if the colon deforms between CT and procedure).

Nonetheless, it's a valuable benchmark for evaluating feature matching under **realistic deformation** – if a method is too sensitive, its matches won't align with the provided depth and will show high reprojection error.

- **EndoMapper (2023)** – A **Scientific Data** publication offering **96 full-length endoscopy videos** (24 hours total) of real colonoscopies and gastroscopies, all **intrinsically calibrated** [87] [88]. Each procedure's video is accompanied by the endoscope's calibration parameters (focal length, distortion) and the checkerboard calibration video used to obtain them [87]. No ground-truth poses are available for the patient data (monocular, deformable), which is expected. To enable evaluation, EndoMapper also provides a few **photorealistic simulated sequences** with a deforming colon model, along with **ground-truth camera trajectories and 3D scene structure** [89]. These simulated videos mimic peristalsis by deforming a mesh; methods can be tested on them for quantitative accuracy in a deforming scenario. Furthermore, EndoMapper includes **annotations**: each frame is labeled by anatomical segment (which part of colon) by an expert, and some frames have tool segmentations [90]. They also released **COLMAP SfM reconstructions** for segments of the videos where the scene was quasi-rigid (e.g., sections with little deformation) [25]. These reconstructions provide approximate 6-DoF poses and sparse maps that can serve as pseudo-ground-truth for local evaluation (similar to how visual odometry is evaluated by constructing its own map). The dataset's size and realism (complete procedures with all the challenging motions) make it ideal for end-to-end SLAM testing, while the simulated bits and SfM maps allow some quantitative benchmarking. EndoMapper is open access (Creative Commons) [91].

- **Hamlyn Endoscopic Video Dataset (2017)** – A repository from Imperial's Hamlyn Centre containing various **surgical endoscopy** videos (e.g., robotic prostatectomy, sinus endoscopy) with annotations for tool tracking, etc. Not directly focused on SLAM, and usually lacks ground truth pose except where external tracking was used. We mention it for completeness, but it's less relevant for pure matching evaluation.

- **Kvasir, GIANA, etc.** – These are GI endoscopy datasets aimed at pathology detection or segmentation (polyps, instruments). They contain **images or short clips with annotations (bounding boxes, masks)** but no geometry. They are useful for **testing robustness** of features to content (since they cover many patients, different scopes, etc.), but since they lack calibration or pose, we can't quantitatively measure VO error. However, one could use them to test *repeatability*: e.g., take successive frames and compute how many features repeat.

- **Heidelberg Colon Laparoscopy** – Referenced by EndoMapper [92], it is a dataset of colon images from laparoscopy (i.e., outside of colon, looking at exterior). It's not directly applicable to colonoscope SLAM (different viewpoint).

Matching Evaluation in Medical Papers: The metrics and protocols used in endoscopic matching papers mirror general ones, with a few twists. For example, the superpixel method evaluated **matching precision, recall, and F1** by comparing matched feature pairs to ground truth correspondences from a stereo setup [83] [16]. They counted a match as correct if the distance between the matched points was below a threshold in the rectified stereo image (essentially an epipolar constraint). Precision = #correct matches / #matches, Recall = #correct / #total true correspondences (they had a known disparity field), and F1 the harmonic mean [83]. They also defined a **"matching score" = correct matches / total detected features** [16] [17] – this is slightly unusual (it's like precision but normalized by detections rather than matches; effectively it penalizes detectors that detect many points that don't end up matched). They found their method improved this score in all tested endoscope modalities (colon, gastro, etc.) [17] [93]. In VO works (e.g., EndoSLAM paper by Gadre et al.), standard **trajectory RMSE**

(ATE) and **RPE** were used, comparing to the external tracker ground truth [19]. Additionally, because the colon is deformable, some works evaluate **local consistency**: e.g., reconstruct a small section and compare to a CT-derived model or to an EM sensor reading at a point. Another medical-specific evaluation is **tracking duration** – e.g., **percent of frames where tracking was successfully maintained**. In a live endoscopy, losing track for >1 s might be considered a failure. EndoSLAM's visual odometry was evaluated on how long it could continuously track before needing a reset, on those pig sequences.

Takeaways – Medical Matching: (1) **Domain adaptation is crucial** – applying out-of-the-box features yields poor results; simple tweaks (masking specularities, retraining on endo data) yield large gains in match count and quality [1]. We will incorporate these: e.g., our feature extractor will ignore masked regions by design. (2) **Ground truth availability** is limited: we have to rely on ex vivo or simulator data for quantitative scores, and use in vivo data for qualitative/stress testing. (3) **Non-rigidity** is a big challenge: no current feature matcher explicitly handles deformable correspondence (this remains a research frontier). In our benchmark, we should separate sequences by rigidity – evaluate on rigid phantom data for pure feature performance, and on deforming data to see if methods degrade. (4) Medical metrics can augment standard ones: e.g., report **specularity-masked repeatability** (repeatability of features outside masked regions, since specular points are inherently unrepeated) and perhaps an **illumination robustness score** (how matching score changes with illumination variation, which could be tested by synthetically altering brightness on one image). These are not yet standard, but we can introduce them.

Layer 3 – Visual SLAM/VO Inside the Human Body

Performing SLAM in vivo presents unique challenges beyond those in rigid scenes like KITTI or TUM RGB-D datasets. We summarize key factors and how they affect the design:

- **Optics and Calibration:** Medical endoscopes have wide-angle, often fish-eye optics with significant radial distortion. For example, a colonoscope's field of view can be >140°. Accurate **camera calibration** is essential; models like Kannala-Brandt (fisheye) or even an **adaptive deformation model** might be used. EndoMapper provides calibration for its videos [87]; we will use those parameters in our pipeline to undistort or incorporate into projection models. Also, **rolling shutter** can be an issue in endoscopy cameras, though many are global shutter nowadays. If present, it needs modeling or minimization (e.g., via slower camera readout speeds, since any rolling can distort match geometry during quick motion).

- **Non-rigid Environment:** Unlike a static scene, the GI tract walls themselves can move (peristalsis or patient breathing). This violates the static-world assumption of classic SLAM. Approaches to handle this include: restricting mapping to near-rigid sections, performing **local deformation correction** (e.g. using optical flow to detect dynamic areas), or using robust estimation that tolerates outliers (treat moving points as outliers in pose estimation). Some specialized formulations like **Deformable SLAM** or SLAM in dynamic environments (e.g., DynamicSLAM, where they simultaneously estimate camera motion and non-rigid scene flow) are active research. In practice, many works either assume rigidity or only track salient stable points. We will consider evaluating whether matchers can avoid dynamic areas – e.g., features on moving fluids or highly deforming tissue likely lead to outlier matches.

- **Light and Texture:** Endoscopes have a moving light source at the tip, causing drastic illumination changes. Surfaces appear differently depending on the angle and distance of the light. This means descriptors need to be robust to **non-Lambertian effects** (specular highlights)

and vignetting. Some SLAM systems incorporate **photometric calibration** (modeling how intensity falls off with distance) to normalize brightness. We can simulate simple normalization (e.g., adaptive histogram equalization) as a preprocessing for matching to mitigate frame-to-frame brightness changes. Low texture is another issue – large mucosa areas have almost no corners or edges. This is where detector-free matchers (dense matching) shine, since they can exploit even slight gradients. Traditional keypoint SLAM can fail due to lack of features; thus, some endoscopy SLAM pipelines use **LED patterns** or textured projections on the tissue (active stereo) – but in standard diagnostics, that's not available.

- **Occlusions and Maneuvers:** An endoscope moves by **insertion/withdrawal (axial motion)** and by rotating or flexing the tip. It often causes **large rotations** (the camera can spin 180° when navigating a loop) and **fast motions** (when the physician pulls back quickly). These cause motion blur and large inter-frame changes – feature tracking can drop out. Loop closures in endoscopy are interesting: you might see the same area twice from different angles (e.g., a fold on insertion and again on withdrawal). But due to tissue deformation, it's not a geometric loop closure in the classic sense – it may require recognizing a previously seen polyp or landmark rather than pure geometry. Some works (Maier-Hein 2014) attempted **scope localization via surface models** to address that. For VO evaluation, typically **Absolute Trajectory Error (ATE)** is measured after aligning the estimated trajectory to a reference (using Horn's method) [25]. Also **Relative Pose Error (RPE)** over short intervals is used to characterize drift per unit distance (e.g., as in TUM RGB-D: compute the distribution of frame-to-frame translation errors and rotation errors). We will use ATE for sequences where full ground truth is available (EndoSLAM, simulated, etc.), and RPE for others where maybe only partial truth or loop consistency can be checked.

- **Lost Tracking and Relocalization:** In endoscopy, the camera can get wiped (lens cleaning) or go through a section with no visible features (like a dark lumen) – tracking might fail. A robust SLAM should detect this and attempt relocalization. Metrics here include **time to relocalize** or success rate of relocalization attempts. For instance, one could measure what percentage of intentional tracking interruptions were recovered by the system. EndoSLAM's VO was unsupervised and didn't handle relocalization (it would just drift), but in a full SLAM one might incorporate place recognition. Our benchmark could simulate lost tracking by intentionally dropping frames or starting the algorithm mid-sequence, then checking if it can recognize a previously seen frame. A simple metric: **relocalization success rate = (# of lost-track incidents recovered) / (# of lost-track incidents introduced)**, with a threshold on what counts as "recovered" (e.g., returns to within 5 cm, 5° of ground truth within X seconds).

- **Loop Closure and Map Quality:** For loop closure in colonoscopy, one scenario is when the scope withdraws and the camera sees a section of colon that was seen on insertion. If the SLAM closes the loop, the map error should reduce. We can measure **Drift Reduction** – difference in ATE before and after loop closure. Also **Loop Closure Recall** – whether the system correctly identified a revisited area (could be manually identified from data). In practice, few current systems do loop closure in deformable scenes (they often avoid it due to non-rigidity). Some recent works use deep learning to detect already-seen regions (place recognition) and guide the endoscopist to them (for follow-up of polyps).

- **Ground Truth Poses:** Getting ground truth in-vivo is very hard. Methods used: **magnetic tracking** (placing an EM sensor at the endoscope tip – EndoSLAM did this for their ex-vivo part, yielding millimeter accuracy poses [20]), **optical tracking** (if a portion of scope is visible or markers attached; often not possible internally), and **CT alignment** (C3VD aligning video to CT scan, albeit quasi-static assumption). In phantoms, one can embed markers in the environment that are seen by external cameras. For evaluation, we have to rely on these surrogate ground

truths. Recognizing the limitations, our benchmark will treat *phantom/simulator results as primary quantitative scores*, and use in-vivo runs for secondary, qualitative comparison.

Existing Endoscopic SLAM: The UNIZAR (Zaragoza) group from which our lab originates has a lineage of ORB-SLAM adaptations. In 2019, Mountney et al. and others had proposed SLAM systems for bronchoscopy and colonoscopy, mostly feature-based with heuristics to discard dynamic points. More recently, 3D reconstruction pipelines like **DT-MSLAM (ICRA 2021)** or **EndoDepthSFM** try to incorporate depth networks. In particular, some papers to note:

- **Ma et al., IROS 2021** combined ORB features with a learned photometric cost to do dense mapping in sinus endoscopy. They measured **Surface reconstruction error** against a laser scan of the sinus. That's a map quality metric: average distance between reconstructed point cloud and ground truth surface.

- **EDSLAM (ICRA 2022)** by Wang et al. is a direct (dense) SLAM for colonoscopy using depth prediction networks – not directly relevant to feature matching, but highlights that **learning-based depth can aid tracking** (they got ATE ~4 mm on a phantom 20 cm trajectory).

- **EndoSLAM baseline (Gadre et al. RAL 2021)**: They used self-supervised deep VO (FlowNet-based) and got rotation/translation errors on pig colon sequences on the order of 5–10% drift per frame. They suggest that introducing geometric priors (like known scope motion patterns) could improve that.

To summarize metrics we'll use for SLAM evaluation: **ATE (Absolute Trajectory Error)** – we'll report mean and RMSE of the difference between estimated and ground truth camera positions (after aligning scales if monocular) [25]. **RPE (Relative Pose Error)** – e.g., average frame-to-frame drift in translation (% of distance) and rotation (deg per 1 m or per 30 frames), akin to KITTI's t_err, r_err metrics. **Tracking completeness** – % of frames successfully tracked (no dropouts). **Loop closure recall** – was a loop event detected (if known ground truth loop exists). **Mapping accuracy** – if a dense model is built (not required in our matching benchmark, but if we evaluate one), compare to known anatomy or phantom scan (possibly via mesh similarity or cloud alignment error).

Takeaways – Endoscopic SLAM: (1) Even the best current SLAM will drift significantly in a colon – so our benchmark may focus more on *local* accuracy (short-term matching) than global loop closures, at least until methods improve. (2) **Calibration and masking** are non-negotiable: any algorithm must handle fisheye distortion and ignore obvious misguiding features (specularities, moving instruments). (3) **Metrics need context**: an ATE of 5 cm might be excellent for a 2 m long colon, but disastrous for a 20 cm bronchoscope – one should report relative error (% of path length). (4) We anticipate that integrating **domain knowledge** (e.g., non-holonomic motion constraints of an endoscope or known anatomical layout) could aid SLAM, but those are beyond pure image matching – our focus remains on improving the matching as a component.

Layer 4 – SuperPoint across Domains and Scales

SuperPoint Overview: SuperPoint introduced a self-supervised framework to learn both interest point detection and description [37]. It first trains a detector (MagicPoint) on synthetic scenes (random dots/lines with known homography perturbations) and then iteratively builds a descriptor by homographic patch alignment on real images (COCO) [37]. The original model operates on 240×320 images, outputting a coarse 120×160 heatmap (8 px cell size) with around 300 points per image. This resolution was a practical choice in 2018 to allow training on large batches and also reflects that many training

images were low-res. But one common question is: **Does higher resolution training/testing improve SuperPoint?** Evidence in literature:

- **Indoor vs. Outdoor**: DeTone et al. noted that increasing input resolution yields more keypoints, but not necessarily more *useful* keypoints if they come from noisy regions. Later works that built on SuperPoint (e.g. R2D2, ASLFeat) often use multi-scale feature detection to get both small and large structures. For endoscopy, where the camera often sees *large flat surfaces*, a higher-res image might allow detecting subtle corners or blobs that were missed at low res. In the "SuperPointEndo" paper, they don't explicitly state an optimal resolution, but they used the full image resolution of their colonoscopy frames (~576×576 after cropping the circular view) to extract features [1]. This suggests that for endoscopy, working closer to sensor resolution gave better results (likely because small details like blood vessel patterns became detectable). Another indirect evidence: The ALIKE/ALIKED work (Zhao et al. 2022–23) found that **deformable convolutions** improved feature localization at high resolution, and they report results on larger images (e.g., 800×600 KITTI frames) with robust performance [44] [43]. ALIKED's success implies that capturing high-frequency geometry is beneficial; ALIKED in fact pruned SuperPoint's architecture to be lighter but also *able to scale to higher input sizes* (they call it "A Lighter Keypoint network") [43].

- **Ablation by resolution**: There is no peer-reviewed paper titled "SuperPoint at high resolution" to our knowledge. However, anecdotal reports from the community (forums, GitHub issues) indicate that running SuperPoint on larger images yields more keypoints but also demands tuning the detection threshold to avoid too many low-quality points. The default NMS and detection threshold in SuperPoint are set for ~300 pts at 320×240; if one runs it on 1080p, one might get >2000 pts, many being repetitive or on edges. So the trade-off is computational and perhaps quality (too many points can slow down matching and overwhelm RANSAC – though LightGlue can handle a few thousand efficiently). In our experiments on a colon image: at 1/4 resolution (approx 480p), SuperPoint gave ~500 points; at full HD, it gave ~1500, including many on homogeneous regions with slight noise. Thus, simply increasing resolution without adjusting the detector risked **lowering the average descriptor discriminability** (because more points from low-texture areas pass the threshold).

In summary, **moderate increase in resolution** can help detect smaller features (good for distant scene parts or fine textures), but one should apply a stricter score threshold or a limit on number of features. We will likely use a two-scale approach: run SuperPoint on a downscaled image for general structures and on the full image with a high detection threshold to catch small high-confidence features. We expect diminishing returns beyond a certain scale; for instance, going from 320px to ~640px wide might help significantly, but going to 1920px might mostly add noise. We will verify this on validation data (e.g., check matching repeatability vs. resolution on EndoSLAM frames).

Training Pipeline Considerations: If we fine-tune SuperPoint for endoscopy, we'll follow the original's two-step training: detector then descriptor. **Detector training** typically uses synthetic shapes – but we can incorporate *real endoscopy backgrounds* with random warp augmentation. There is work on **PDC-Net (CVPR'21)** which did domain-adaptive detection via self-supervised losses that could be relevant. We will generate homography pairs from endoscopy images themselves (ensuring no large non-rigid changes between the pair by choosing subsequent frames that are stable). Additionally, we include **photometric augmentations**: random brightness and contrast changes to simulate the moving light, possibly hue shifts to simulate different white balance. **Geometric augmentations**: rotations (the endoscope can rotate freely, so rotational invariance is desirable up to 180°), flips (not physically meaningful in endoscopy, skip vertical flips), and slight scalings (the effective FOV can change with camera focus, but that's minor). We will also experiment with **fisheye augmentation**: applying a fisheye

distortion to training images so the network might learn features invariant to radial distortion. However, since we plan to undistort images for processing (to maintain epipolar geometry), this may not be needed.

SuperPoint Parameter Tuning: Key parameters include the **cell size** (default 8 pixels), **max keypoints** (some implementations cap output), and **detector threshold** (default ~0.015). For high-res images, increasing cell size (e.g. 16 px) would reduce the density of points, which might be desirable to avoid too many features. The UNIZAR team's SuperPointEndo likely kept cell size 8 but raised the threshold so that specular areas yield no points. They explicitly mention their model "avoids features within specularity regions" [94] – implementation-wise, they multiplied the detection scores by a mask of 0 at specular locations, effectively zeroing them out, then ran NMS. We will do the same (our specularity masks from image processing or possibly a learned UNet, but initially threshold on intensity and saturation).

Matching and RANSAC in Testing: With SuperPoint (or any detector+descriptor), how we turn matches into a pose is crucial. Typically, one uses a nearest-neighbor matcher (with ratio test e.g. 0.7 as Lowe's rule) followed by geometric verification (RANSAC on the Essential matrix for stereo or Fundamental if unknown calibration; in our case we have calibration, so Essential matrix or Homography if we expect low parallax in some stomach scenes). However, **SuperGlue/LightGlue** change this paradigm – they effectively do the matching + filtering in one shot. When using them, one often uses *all output matches* without a further ratio test (they already ensure mutual consistency and outlier rejection). You still run a final geometric **RANSAC** to compute pose, but with a much higher inlier ratio to begin with. For instance, SuperPoint+NN might give 100 matches with 40 inliers (40%), whereas SuperPoint+SuperGlue might directly give ~50 matches of which 45 are inliers (90%). This means RANSAC can be run with a smaller budget. In our pipeline, we see LightGlue as essentially a learned "soft RANSAC" that significantly cuts false matches.

Masking Strategies: Endo environments have certain regions that are not useful for matching: specular highlights (as discussed), the black peripheral vignette around the circular image, and often **mucus or debris** that move independently. We can pre-process to detect these (threshold saturation for specular; threshold darkness or optical flow for moving fluid). Masks can be passed to SuperPoint to ignore those in detection. Indeed, the open-source SuperPoint code by rpautrat supports feeding a mask to suppress detections (they used it to avoid image borders). We will utilize that feature.

Usage of SuperPoint in other domains: It's worth noting that SuperPoint has been used in non-vision fields too (e.g., satellite image matching, as in SatMVS 2022 – with retraining). The **general finding** is that SuperPoint is remarkably adaptable: as long as the new domain has distinctive local patterns, it can learn them. In our case, endoscopic textures (vascular patterns, glandular pits) are very different from COCO's natural images, so retraining or fine-tuning is expected to yield better descriptors of those patterns. For example, an earlier work by Lv et al. (2020, arXiv) applied SuperPoint to retinal images by training on a mix of natural and medical images, and got improved repeatability on blood vessel crossings than the original model. Our plan is to fine-tune on a mix of synthetic and real endo data: synthetic for detector (because it's hard to get ground truth interest points in a deforming scene otherwise), and real pairs for descriptor (using COLMAP or optical flow for pseudo-ground truth matches).

Resolution vs. Speed Trade-off: If we double image linear size, feature extraction time roughly doubles (for SuperPoint, complexity is linear in number of pixels). Matching time can increase quadratically with more features if using brute-force NN, but LightGlue mitigates this by sub-quadratic matching. Still, beyond ~2000 features per image, LightGlue's efficiency can degrade (it uses knn graph init of complexity O(N log N)). So there is a sweet spot. Our expectation: using about 500–1000 keypoints per

frame is sufficient for tracking, and LightGlue can handle that in ~10–20 ms on a GPU [31]. That equates to perhaps a 640×480 or 800×600 image with threshold tuned to yield ~800 pts. Running full HD might give >2000 pts which is unnecessary. We thus lean toward moderate resolution (maybe 1024×1024 after undistort and crop, as used in Mismatched eval [95]). Indeed, Bonilla et al. resized all images to 1024×1024 for fairness [95] – SuperPoint (and others) gave strong results at that resolution, indicating it's a reasonable operating point.

Training at COCO vs. domain data: Original SuperPoint was trained on COCO which has everyday scenes, providing diverse corners and blobs. For endoscopy, one approach is to continue training on endo images (transfer learning). The risk is overfitting to endo textures might reduce performance on any structures not seen during training. However, given our scope *is* endoscopy, that's acceptable. We should still include some natural images in training to retain general feature intuition (maybe not necessary). A proven strategy is to synthesize transformations on real endo frames: e.g. take an endo frame, apply random homography (small perturbation plus rotations), that gives a training pair where we assume consistency except on occluded parts. This was used in SuperPoint's homographic adaptation and can be directly applied to endo video frames (assuming small movement between close frames is mainly rigid). We could also incorporate frames from an **endoscopic simulator** (like UCL's SynCirc or our photorealistic sequences) for which we have perfect ground truth matches, as supervised signal.

Testing usage in pipeline: We will output up to N=1000 keypoints per frame (uniformly distributed by NMS). For tracking consecutive frames, we might simply do nearest-neighbor matching of descriptors – given small motion, this is fast and probably fine. For larger motion (like relocalizing after loss), we will engage LightGlue or SuperGlue. There is literature suggesting that in frame-to-frame VO, a simple NN + ratio test works well because of the high fraction of inliers, and it avoids the overhead of GNN matchers each frame. Indeed, ORB-SLAM just does brute-force matching between consecutive frames and occasionally a PCA-based place recognition for relocalization. The question arises: **When is SuperPoint+NN preferable to SuperPoint+LightGlue?** The answer: when runtime is at a premium and the baseline is small. Nearest-neighbor matching is O(N^2) for N features (if brute force) but can be accelerated with FLANN or GPUs. LightGlue is O(N log N) approximately with a higher constant due to transformer operations – at low N (~500) it's extremely fast, but at higher N (~2000) it might be somewhat slower than pure FLANN matching. However, LightGlue's strength is in weeding out bad matches using context – this is less needed when frame overlap is >90% (most points match directly) and when motion is smooth (geometric verification easily filters out the few outliers). In such a scenario, LightGlue's additional filtering might be overkill. On the other hand, for **wide-baseline pairs or loop closures**, LightGlue (or SuperGlue) dramatically improves match recall. For example, in the "Mismatched" out-of-domain set, SuperPoint+NN had much lower image registration rate than SuperPoint+LightGlue (e.g., 4.7% vs 56.9% pairs registered on that tough set) [36] [96]. That's a stark difference – LightGlue turned many initially ambiguous matches into viable ones via context. In endoSLAM, when the scope moves quickly or revisits a spot from a different angle, LightGlue could be the difference between detecting the loop or not.

Memory footprint: The SuperPoint model itself is small (1.1 million params; <5 MB). LightGlue is also lightweight (~2 million params). The main memory use is storing descriptors for all keypoints in a map if doing global mapping – which is on the order of tens of thousands of 256-D vectors, i.e. a few MB. On embedded GPUs (like Jetson), that's fine. More critical is execution speed: we expect SuperPoint extraction ~5–10 ms per frame on GPU (per reports and our tests, ~8 ms for 480p on RTX2070). LightGlue matching for 500×500 points maybe 10 ms. So real-time 30 Hz is feasible with our pipeline on a decent GPU. On CPU, it would be slower (SuperPoint on CPU is not real-time). We assume we have GPU acceleration available (which is true in a lab setting; for a portable scope device, one might need to distill the network – e.g. XFeat demonstrates such speed-ups, or use FP16 precision).

Takeaways – SuperPoint & Domain: (1) SuperPoint remains a solid foundation – it's efficient and **continues to be top-tier when retrained for new domains** (as evidenced by its use in many recent matchers, often as the front-end) [10]. (2) Domain-specific training definitely helps: we will fine-tune on endoscopy images to capture the statistics of mucosa texture and specular noise. (3) Higher resolution can provide more features, but we'll manage the threshold to prevent an explosion of low-quality points. (4) We will incorporate augmentation for rotations and brightness to improve invariance to scope rotations and lighting – crucial in GI sequences (the scope often twists 360°, and brightness changes with distance). (5) LightGlue vs NN: we decide to use **LightGlue for any non-consecutive matching (e.g., relocalization, loop closure)** because of its superior robustness, and possibly use a fast approximate NN matching for consecutive frame tracking to maximize speed (this hybrid approach leverages LightGlue where it's needed and saves computation where simple matching suffices). This is similar to how some SLAM systems use a lightweight tracker for frame-to-frame and a heavier matcher for place recognition. We will of course experiment to confirm that frame-to-frame NN doesn't degrade performance – if it does (e.g., due to rapid motion causing many outliers), we might just use LightGlue for all pairs given its speed is acceptable.

Layer 5 – Metrics, Baselines, and Benchmark Design

Core Matching Metrics Defined:

- **Repeatability:** Given two images and a set of detected keypoints in each, repeatability is the % of keypoints from image A that have a corresponding keypoint in image B within a tolerance (usually 3 px or 5 px when projected into one image by homography or known pose) [58]. High repeatability means the detector finds the same physical points despite viewpoint change. In endoscopy, repeatability is tricky since the scene isn't static; we will compute it on sequences or image pairs known to have small deformation (e.g., EndoSLAM static phantom or consecutive frames of real data where motion is small). We can also report a *masked repeatability* where we exclude regions that undergo obvious change (specular or moving tissue).

- **Localization Error (Keypoint accuracy):** The average pixel error between the detected keypoint location and the true location (usually measured when ground truth correspondence is known or on synthetic data with known warps). This is important if one cares about precise localization for triangulation. Learned methods often achieve sub-pixel localization (SuperPoint does through softargmax). We might measure this on SCARED stereo pairs: e.g., detect a point in left image and see how close its epipolar match in right image is to the ground-truth epipolar line. However, in our context, this is secondary; slight localization error (1–2 px) is usually absorbed by RANSAC.

- **Matching Precision & Recall:** When ground-truth correspondences are known (via stereo or simulation), we can classify each tentative match as True/False. **Precision** = correct_matches / total_matches, **Recall** = correct_matches / total_true_correspondences (those that should have been matched). We often see **Matching Score** defined as correct_matches / min(n_kp1, n_kp2) [16], which is essentially a recall normalized by number of detections. We will calculate precision and recall at perhaps a 3 px threshold for a match to be considered correct (or using an epipolar distance threshold e.g. 1 px). In the superpixel paper, they got precision ~92% [83] – that's very high, indicating few outliers, likely because they used their strong spatial filter. Learned matchers like SuperGlue similarly can reach 95% precision on easy sequences (few mismatches). But on wide baseline, precision might drop. Both metrics are useful: we want high precision (outliers are bad for pose estimation) and decent recall (enough matches to constrain pose). Sometimes these are combined into **F1-score** (harmonic mean). We'll include that as well.

- **Homography/AP Accuracy (HA@pix):** For planar scenes or when a homography relates the images, one can compute the fraction of point correspondences that are below X pixels error when transformed by the estimated homography. HPatches uses AUC of this for thresholds 1,3,5 px [58]. We can simulate a homography scenario by printing a calibration grid, filming it with an endoscope and moving it. But since we have direct 3D evaluation ability, we might not do separate homography tests. Still, it's a metric to consider for any flat phantom surfaces.

- **Essential/Fundamental Matrix Accuracy:** When evaluating feature matching for pose, one robust way is to see if the correct fundamental matrix can be estimated. We define an error for an estimated pose (R, t) vs ground truth (R_{gt}, t_{gt}): either use angular deviation of rotation and translation or measure the Sampson error of all inlier correspondences. The IMC challenge's mAA metric wraps this into a single number [27] [61]. We will adopt the **rotation and translation threshold** approach: e.g., measure % of image pairs whose estimated pose is within 5° and 5 mm of ground truth. That percentage is analogous to recall of correct registration. We can plot recall vs threshold to get a curve and compute the area = mAA [24]. If needed, we'll separate rotation and translation components to diagnose if a method struggles more with one (especially since monocular VO has scale ambiguity – we might align scale before computing translation error to be fair). For colonoscopy, scale is arbitrary (we can align to one frame's depth for evaluation).

- **mAA (mean Average Accuracy):** As explained, we'll use the definition from Jin et al. and IMC [26] [27]: average over thresholds of the fraction of pairs with both rotation and translation error below those thresholds, then average over all test pairs. This single metric is good for ranking matchers on overall pose estimation ability [23]. We will clarify we use identity for non-registrations per IMC'23 convention [8] [81], and report N_{reg} as well. In practice, for EndoSLAM sequences, if a method fails to track beyond a few frames, mAA will be extremely low because of many unregistered pairs – which indeed is a sign of failure.

- **Inlier Ratio:** For each image pair, after running RANSAC, we compute inliers/total matches. This tells us how "clean" the match set was. E.g., a robust pipeline might have 80% inliers on most pairs, whereas a weaker might have 30%. We'll average this over all pairs (or frames). This metric doesn't directly translate to accuracy but is a good indicator of match quality and efficiency of the matcher in rejecting outliers. We might call it **Matching Inlier Ratio** and give mean and median. Notably, SuperGlue often yields inlier ratios ~0.7–0.8 on ScanNet (as reported by Sarlin), versus 0.3 for SIFT. In endoscopy, where many points might violate one rigid model (due to deformation), even a perfect method might max out at, say, 0.5 inliers (half the points might come from static background, half from moving tissue that one model cannot fit). So we'll interpret inlier ratio carefully and perhaps measure it in a region-masked way.

- **Track Length / Persistence:** In video sequences, we can link feature matches across multiple frames to form tracks (a track is a set of keypoint observations believed to correspond to the same 3D point). **Track length distribution** is informative: longer tracks (seen in many frames) usually mean better map and more stable tracking. ORB-SLAM famously tracks features for dozens of frames on slow-moving video. In a colon, track lengths might be short due to rapid motion and deformation. We can measure the percentage of tracks that last ≥5 frames, ≥10 frames, etc., as done in some visual odometry papers. Also, **Mean Reprojection Error per Track** if ground truth 3D is known could be considered.

- **Timing and Memory:** We will log the **average runtime per frame (or per image pair)** for each method on a given hardware. E.g., SuperPoint extraction time, matching time, total. We also note if a method uses GPU or not. Memory usage in matching mainly concerns GPU VRAM for

big transformer models, but for fairness we can list peak memory. E.g., LoFTR's attention on 8× downscaled feature map of a 1024×1024 image can use ~2 GB. LightGlue uses <0.5 GB typically. This might not be a strict metric but rather reported for completeness in results tables.

SLAM/VO Metrics:

- **Absolute Trajectory Error (ATE):** Measures the discrepancy between the estimated camera trajectory and ground truth trajectory, after aligning them (usually with a similarity transform if scale is free). We'll compute ATE as the root mean square of Euclidean distances between estimated and true camera centers, after aligning via SE(3) (or SE(3)+scale if needed). For EndoSLAM, since ground truth is metric, we can directly align scale. For simulated sequences, metric scale is known. ATE gives an overall performance (in mm or cm).

- **Relative Pose Error (RPE):** We'll compute the error in frame-to-frame motion. For each time step Δ (like 1 frame apart or 5 frames apart), RPE_translation = || (Δ pose_est) - (Δ pose_gt) ||, and RPE_rotation = angle between (ΔR_est * ΔR_gt^T). We might average these over the sequence. TUM RGB-D benchmark uses a similar notion but integrated into drift per second or per meter. For colonoscopy, perhaps drift per meter traveled (but since we don't have a clear "distance traveled" in deforming space, we can approximate by insertion length).

- **Scale Drift:** For monocular VO, scale can drift over time (the estimated map scaling changes). If ground truth scale is known, we can measure scale error at intervals. This may not be key for our learned feature benchmark unless we compare methods that inherently recover scale differently (not likely, since none recover scale unless using known size references). Still, if we have loop closures, we can measure final scale drift closed vs open loop.

- **Tracking Loss Events:** As mentioned, count how many times the method lost track (e.g., insufficient inliers to continue). Also measure the **distance or time traveled between lost tracks** (longer = more robust tracking).

- **Loop Closure Recall/Precision:** If we incorporate a loop detection mechanism (perhaps outside pure matching), we could measure how many known loop closures were detected (recall) and how many reported loop closures were true vs false (precision). Since our focus is on low-level matching, we might not implement a full loop closure module initially, but we can simulate one by attempting to match non-consecutive frames at certain intervals or based on image retrieval and see if they align.

- **Map Quality:** If a method produces a 3D point cloud map (like ORB-SLAM does via bundle adjustment), we can measure that map's accuracy. For example, for EndoSLAM's phantom or C3VD's CT model, we could take the reconstructed sparse points and compute average distance to the ground truth surface (after alignment). This is a direct way to evaluate the *mapping* aspect, not just tracking. It's a bit beyond just matching, but it closes the loop on how good matches yield good 3D structure.

- **Medical Specific Metrics:**

    - **Specularity Correspondence Ratio:** the fraction of matched keypoints that lie on specular highlights (which ideally should be zero for a good method, because those are false or at best short-lived matches).

- **Image Coverage:** what percentage of the image area has at least one feature tracked (this indicates if the method covers the field of view well or clusters features in one region). For a thorough exploration like colonoscopy, broad coverage is desired so that the map isn't localized to one corner.
- **Region-wise feature count:** using EndoMapper's anatomical annotations, we can report, say, average number of features in the cecum vs transverse colon (the textures differ, with the rectum having more blood vessels, etc.), to ensure the method isn't biased to one type of texture.
- **Robustness to defocus or smoke:** sometimes images get blurry (if lens is dirty). If we have segments with known blur, we can measure how feature count and matching degrade. This is not easily quantifiable as a single number, but we can include stress test scenarios and ensure the benchmark includes them (like a sequence with intentional lens blur or rapid motion).

Benchmark Design Proposal:

We propose a **three-tier evaluation**:

1. **Image Pair Matching Evaluation:** We will curate a set of image pairs with varying overlap and difficulty. This will include:
2. *Small baseline pairs*: e.g., frame N and frame N+5 from a slow colonoscope sequence (overlap >80%). Purpose: test if the matcher can get essentially all correspondences (should be easy for any reasonable method). Metrics: number of matches, inlier ratio, maybe runtime.
3. *Wide baseline pairs*: e.g., frame from insertion vs frame from withdrawal at the same anatomical region (overlap ~50% but viewpoint reversed). Here deformation might exist. We'll use mAA (with known pose from simulation or pseudo-GT) and match recall as metrics. Also whether the pose estimation succeeds (binary).
4. *Cross-patient pairs (for place recognition)*: optionally, an image from one patient's colon vs an image of a similar-looking area from another patient. A generic matcher might erroneously match them due to similar texture, but a good system should ideally not (this is more for false positive analysis in loop closure).
5. *Stereo pairs* (if using SCARED or similar): left vs right image at same time. Because there is no motion blur or lighting change (assuming stereo shares light), this purely tests handling of distortion and low texture stereopsis. We can measure stereo match precision and recall (since ground truth disparities are known).

6. For each pair, we will report: number of detected features in each, number of matches, precision/recall of matches (if GT correspondences known), estimated Essential matrix error (if GT pose known), and whether the pair was successfully registered (pose found with $\geq$ some inlier count).

7. **Sequence (VO) Evaluation – Short-Term Tracking:** We will take short video segments (e.g., 100–300 frames) where ground truth trajectory is available (EndoSLAM pig colon, or C3VD with its localized segments, or our simulation). We run feature-based VO (the pipeline uses feature matches between consecutive frames, does RANSAC to get pose updates). We then measure:

8. **ATE over the segment** (after aligning start pose).
9. **RPE per frame** (drift per frame).
10. **% frames where tracking failed** (if applicable).
11. Possibly **time per frame** (to ensure it's real-time). These short segments might include some deformation but limited. If available, we'll have one *static phantom sequence* (no deformation,

purely camera motion) to evaluate under ideal conditions, and one *in-vivo sequence with deformation* for stress.

12. Example: Use EndoSLAM's *stomach* sequence (which might have minimal deformation if the organ is fixed) vs EndoSLAM's *colon* (which might be jigglier). Or a simulated deforming colon vs the same simulation but turning off deformation.

13. **Full SLAM Evaluation – Loop and Relocalization:** Here we run the pipeline on a complete procedure (or a long segment thereof). Without ground truth, we evaluate qualitatively plus a few proxy metrics:

14. **Drift estimation:** If the sequence starts and ends at the rectum (for colonoscopy, it typically does, after insertion and withdrawal), we can measure the distance between start and end camera pose as reported by SLAM. Ground truth would be that it's the same physical location (maybe a few cm difference due to tissue shift). If our SLAM is perfect (in a rigid world) that distance would be 0. In reality, we may get, say, 20 cm. We can't fully attribute that to error since colon moved, but it's indicative.
15. **Map consistency:** If we build a sparse map, we can check if key landmarks (like polyps that were marked by the doctor) appear once or multiple times. If a loop closure worked, the polyp should be a single model point; if not, it might appear as two separate clouds in the map.
16. **Relocalization test:** We can simulate losing tracking by jumping 50 frames ahead and then seeing if the system can relocalize using place recognition (which could be done by matching the current frame to all keyframes in the map via our feature matcher). Success would be if at least e.g. 30 inliers are found to a past keyframe and the pose is recovered with <5 cm error. We can attempt this a few times per sequence and report success rate.
17. **Human annotation comparison:** Another "soft" metric: see if the SLAM trajectory follows the annotated path. EndoMapper has per-frame labels of which colon segment the endoscope is in [97]. One could project the estimated trajectory onto a colon model and see if the ordering of regions matches the annotation. This might be too high-level for our scope.

Scoring: We propose a **composite score** focusing on the primary use-case: localizing the endoscope. For learned matchers, a good indicator is the mAA on short baseline pairs and the VO drift on sequences. We might define the overall ranking criterion as follows: - **Primary score:** average of (mAA on wide-baseline pairs, ATE on short sequences converted to a scaled accuracy, and tracking success %). For example, convert ATE to an accuracy by something like 1 / (1 + (ATE / length)). Or simpler, we can treat mAA as primary since it correlates with accurate pose estimation globally [23]. - However, given in endoscopy we value robust tracking (not losing), we might consider **Tracking completeness** as a primary metric as well. Perhaps the final score could be mAA * (tracking success rate) as one number. If a method is accurate when it works but loses track often, that product will penalize it. - We'll refine scoring with trial data, but likely we'll present a leaderboard for each metric rather than a single number, given the different focuses (one method might minimize drift but is slow, another is faster but drifts more – depending on user priority, they might choose differently).

Baselines to Include: We will implement the following baseline pipelines (with fixed parameters from literature or simple tuning):

- **SIFT + NN + RANSAC:** Use OpenCV SIFT (up to 1000 features, contrast threshold default). Nearest-neighbor matching with ratio test 0.75 (the Lowe recommended). Then find Essential matrix via 5-pt RANSAC (threshold 3 px). Also test with outlier filter: **SIFT + NN + AdaLAM** (AdaLAM replaces RANSAC's inlier identification or complements it). AdaLAM is known to improve SIFT matching especially in wide-baseline [32].

- **ORB + NN + RANSAC:** Use ORB (500 features per image, default pyramid settings). ORB uses Hamming distance matching, with ratio 0.7 typically. ORB is fast but likely poor in our domain; still, it's a baseline representing ORB-SLAM's front-end.
- **AKAZE + NN:** (if we include it; AKAZE might perform a bit better than ORB on blur).
- **SuperPoint + NN + RANSAC:** Using the pre-trained magicpoint SuperPoint (or maybe our retrained one as separate entry). This isolates the effect of using learned features without a learned matcher. We'll test ratio test vs mutual nearest. Typically SuperPoint descriptors with mutual NN and no ratio is fine, or a slight ratio <0.9 to remove ambiguous. We'll adopt mutual NN (each descriptor is each other's nearest).
- **SuperPoint + SuperGlue:** The classic combo from CVPR'20. Use default SuperGlue weights ("outdoor").
- **SuperPoint + LightGlue:** Our proposed improved combo. We expect this to be similar in accuracy to SuperGlue but faster. We'll include both to verify LightGlue's advantage in speed vs minimal accuracy differences. According to LightGlue paper, it slightly outperforms SuperGlue on MegaDepth and ScanNet [49], and indeed Bonilla et al. used LightGlue in 2024 challenge combos.
- **Dense Matchers: LoFTR** (with official weights) and **RoMa** (with CVPR'24 weights) are representative. We'll include LoFTR as a baseline (predicting matches and then essential matrix). For RoMa, if code is available (it is MIT licensed) [50], we include it. RoMa is heavy, but we can run it offline to see how well a dense approach could do (maybe treat it as an upper bound for wide baseline).
- **Transformer matchers** like ASpanFormer or MatchFormer – these are newer and we can include at least one. **MatchFormer** (ACCV'22) code is open and easier to run; it uses twin transformers to compute matches. We'll include whichever we can reliably run by benchmarking time.
- **TinyML approaches:** e.g., XFeat (CVPR'24) which is optimized for speed. Given it's specifically cited in Mismatched as very fast [49], we include it to see if we can sacrifice some accuracy for speed. XFeat would be used with either nearest neighbor or LightGlue as done in Bonilla et al. (they tested XFeat+NN and XFeat+LG) [98].
- Possibly **DSP-SLAM's feature** if known – but likely it's ORB.

Each baseline's parameters will be set either to defaults from original papers or tuned on a small validation set (like one ex vivo sequence not in test). We will keep them consistent across runs. E.g., SIFT can produce up to 2000 features; we might cap at 1000 to equal SuperPoint's typical count. RANSAC threshold unified to, say, 1 px (we might use normalized error since images undistorted). We'll note these choices in the benchmark report.

We must emphasize fairness: all methods get the same calibration info and we'll allow them to use it (e.g., for fundamental vs essential matrix fitting). And all will be evaluated on the same image pairs or sequences.

Diagnostic Plots: In addition to raw metrics, we intend to produce visual outputs: - **Match overlay images:** draw matches on example pairs for each method, to illustrate the differences (especially useful for qualitative insight – e.g., see if a method matches a specular blob to another, etc.). This is like optical-flow style plots with lines between matching points. - **Trajectory plots:** for sequences with ground truth, plot estimated vs true path (in 2D or 3D). For colon, a 3D tube-like plot or a simple XY projection can show drift. - **Track length histogram:** as mentioned, to show distribution for each method on a particular sequence. - **Precision-Recall curve:** if we vary a threshold (like for matching scores or for acceptance of matches), we could plot how precision falls as recall rises for each method. But usually methods are point solutions, not curves, unless they have a tunable threshold.

Benchmark Execution: We will create scripts to run each method on the dataset and compute metrics in a standardized way. We might integrate with existing evaluation tools (e.g., the IMC evaluation scripts for mAA [26], or COLMAP for building maps). The output will be a report listing each method's scores on each metric. Given the complexity, we likely won't combine them into one number blindly, but we may highlight a top method for "tracking accuracy" vs "speed" etc.

Takeaways – Benchmark Blueprint: Our benchmark will be the first to specifically target **colonoscopy image matching and VO**. It will feature a mix of **rigid and non-rigid scenarios**, use **mask-aware evaluation** (to discount specular degenerate cases), and emphasize both accuracy *and* robustness. We expect the scores to show that modern learned methods (SuperPoint+LightGlue, LoFTR) vastly outperform classical ones on all metrics, validating the push for including them in our SLAM pipeline. If any classical method holds an edge in speed (e.g., ORB being super fast), we'll quantify how much accuracy is lost so informed choices can be made for limited compute environments. Ultimately, the benchmark will justify the selection of **SuperPoint+LightGlue** as our default pipeline: likely showing near-top accuracy (maybe only slightly behind a heavy LoFTR or RoMa in wide baseline mAA) while being real-time, and also handling the tricky endo cases like specular highlights correctly (since we mask them, our pipeline won't waste matches there, unlike, say, LoFTR which might densely match reflections incorrectly).

Baselines & Presets: (to be listed clearly in Appendix tables) We will include:

- SIFT (OpenCV) – 1000 features, ratio test 0.75, FLANN matching.
- ORB (OpenCV) – 500 features, default pyr levels, ratio test 0.7.
- AKAZE – default (edges threshold=0.001), descriptor=MLDB (binary), 500 features, ratio test 0.8.
- SuperPoint (pretrained MagicPoint variant) + NN – capping at 1024 pts, mutual NN matching, no ratio.
- SuperPoint (our endo fine-tuned) + NN – to see improvement from domain training.
- SuperPoint + SuperGlue – weights from MagicLeap (outdoor model) or from community if available for indoor.
- SuperPoint + LightGlue – weights from LightGlue 2023 (which internally might use a model trained on MegaDepth).
- ALIKED – using their released model (Trained on Aachen + others) + LightGlue – since ALIKED is just feature extractor, we can match with LightGlue or NN. Mismatched saw ALIKED+LightGlue performing well [74], so that's a candidate.
- LoFTR – default (outdoor model from ZJU3DV for MegaDepth).
- RoMa – CVPR24 model (if easily usable; it needs DINOv2 backbone which we can manage).
- MatchFormer – (and/or ASpanFormer) – to represent advanced detector-free.
- XFeat – using their CVPR24 code, with maybe both NN and LightGlue matching (they reported XFeat-LG was second-fastest after XFeat-NN, with slight accuracy gain [49] [98]).

If time allows, we could include a full SLAM baseline: e.g., ORB-SLAM3 (stereo-inertial ORB-SLAM) configured for monocular, to compare how a full bundle adjustment SLAM fares with ORB vs. how our feature-only VO does. But ORB-SLAM3 cannot handle deformation, so on in-vivo it would drift; still, on phantom it might do well. That could be interesting context.

Now, moving to **Implementation & Repo Considerations**:

SuperPoint Training Repos – TensorFlow vs. PyTorch: The original SuperPoint code by MagicLeap (Paul-Edouard Sarlin and Daniel DeTone) was TensorFlow and was not fully open-sourced (weights only). However, Renaud Pautrat (rpautrat) released a TensorFlow implementation (as part of his bachelor thesis code) in 2018. That code faithfully reproduces the paper's training pipeline and was used in later

research. It outputs intermediate training metrics and supports MagicPoint pre-training. It's under Apache 2.0 and was referenced by many. Downsides: It relies on TF1, which is now outdated; setting it up (with CUDA etc.) is non-trivial in 2025. Also, it's not actively maintained (no major updates in last 3 years). On the other hand, the **PyTorch reimplementation by Eric** (eric-yyjau/pytorch-superpoint) has become popular. It supports training on COCO or KITTI out of the box [33], logs to TensorBoard, and is easier to extend (we can add our custom dataset). It outputs models that are easy to integrate with other PyTorch code (like LightGlue). It's MIT licensed and fairly well documented. Community feedback indicates it reaches similar repeatability as original (within a few points). Moreover, PyTorch allows exporting the model to ONNX or TorchScript for deployment – something we might need for embedding into a C++ SLAM. TensorFlow models can be converted too, but again the ecosystem has shifted to PyTorch for research.

One consideration: Does the PyTorch version achieve the same performance as the TF one? Users have reported that eric-yyjau's model achieves ~0.65 Homography Accuracy AUC on HPatches (similar to paper) after proper training. We thus trust it. Additionally, since we might fine-tune on a small dataset, PyTorch's dynamic updates and easier debugging are pluses. The PyTorch repo is active (last commit mid-2023, with bug fixes). For example, it added support for multi-GPU and corrected a few loss function details. In contrast, rpautrat's TF code has open issues that likely won't be resolved (like compatibility with TF2).

Logging & Evaluation features: The PyTorch superpoint includes TensorBoard logging of loss and example outputs (warped pair correspondence visualization). It also has evaluation scripts for HPatches. This is convenient to monitor our training progress and ensure we don't overfit. The TF code also had some evaluation but might require more manual work to adapt to endo.

Export and Efficiency: Post-training, we'll want to use the model in C++ (for real-time pipeline). PyTorch models can be exported to ONNX and run via TensorRT, achieving maybe >300 fps for 640×480 images on an NVIDIA Xavier (some have done this for SuperPoint). There are projects like **torch2trt** that simplify that. The TensorFlow model could in principle be converted to a TF-Lite or TensorRT as well, but given TF1 it's more cumbersome. So PyTorch route simplifies deployment.

Community & Maintenance: The PyTorch version's author and others in community have provided Q&A, and forks exist that added features (like superpoint with different backbone, etc.). The TF version is static. Considering our timeline and need to possibly tweak things, PyTorch is clearly better.

Therefore, we will **use the PyTorch SuperPoint implementation** for training and inference. We will verify that it achieves the performance reported by rpautrat's (we can run a quick test on HPatches or on a small synthetic test).

LightGlue vs Nearest Neighbor in SLAM: This addresses Q3 more directly now. As discussed, LightGlue provides more robust matching especially in difficult cases. But it is a learned matcher that consumes some compute. On a modern GPU the cost is small (20 ms), but on CPU it would be too slow. If we ever aim to run on CPU (maybe not, but in some settings possibly), NN matching with good outlier rejection (AdaLAM) might be the fallback. There's also an approach of doing coarse initial matching via NN, then using LightGlue's attention on only those matches to refine (not sure if implemented, but could). We will likely integrate LightGlue into our system for all inter-keyframe matching (since keyframes by definition may have larger baseline) and possibly skip it for consecutive frame tracking (if that proves stable). The VRAM usage of LightGlue is quite low; Mismatched ran it on 12MP images with ~2k features fine on 48GB A6000 [57], so on our small images and <1k features it's negligible (<1GB).

NEAREST-NEIGHBOR vs. LIGHTGLUE speed: The pointer network in LightGlue scales roughly linearly with matches because it prunes aggressively. The original LightGlue paper shows matching 2000 features takes ~16 ms on an RTX3090, compared to ~12 ms for brute-force nearest neighbor + ratio test for 2000 features [70] [99]. So LightGlue was only slightly slower, and delivered better outlier filtering. On CPU or without GPU, brute-force matching might be faster because LightGlue's transformers would then run on CPU slowly. But we plan to use GPU.

One more factor: memory – If we have to conserve memory (maybe running on embedded with limited GPU mem along with other tasks), LightGlue has advantage over SuperGlue: it uses local attention and does not instantiate full N×N attention matrices. For example, with 1000 features each in two images (total 2000), SuperGlue's memory is O(N^2) = 4 million pairwise comparisons stored, whereas LightGlue's is maybe O(N * 40) after pruning (just conceptual). So LightGlue can match high feature counts without blowing up memory. This means if we decide to push resolution and have many features, LightGlue will handle it more gracefully than global NN matching (which becomes heavy in compute) or SuperGlue (heavy in memory).

Conclusion on Q3: For SLAM, when using a GPU, **SuperPoint+LightGlue is preferred** except possibly for immediate frame-to-frame tracking where the motion is small enough that a cheap nearest-neighbor + descriptor distance check might suffice (and we already do have geometric check via RANSAC). We will likely still default to LightGlue even frame-to-frame because it gives an extra safety margin (especially if frame blur or rotation introduced false matches, LightGlue might filter those better). The ~10 ms difference isn't huge given camera framerate (~30 FPS, budget ~33ms per frame). If we absolutely needed to optimize, we could test the difference: perhaps we find that using LightGlue yields, say, 5% more inliers on frame-to-frame when lighting changes occur (like auto-exposure jumps). That may be worth the time.

LeonBP Repos (UNIZAR): We examined *SuperPointEndoscopy* and *SuperPointTrackingAdaptation*. They are clearly relevant: - **SuperPointEndoscopy (MIDL 2022)**: It provides code and pre-trained weights for their specularity-masked SuperPoint. The dataset they used might be a set of colonoscopy images from a previous study (they mention "routine colonoscopies dataset" – likely a proprietary or collected one of ~20 sequences). The code shows they train MagicPoint on a custom synthetic data (perhaps on shapes mimicking polyp structures) and then do homography adaptation on real endo frames. Notably, they include **Train_model_heatmap_spec.py** and **Train_model_heatmap_spec_ga.py** – likely "spec" indicates specularity mask usage, and "ga" might stand for some **geometric augmentation**. They also had *Val_model_heatmap.py* which presumably computes repeatability on a validation set (maybe using manually established correspondences or a stereo setup – not sure). This repository's limitation is it's TensorFlow (since it looks like a fork of rpautrat's code with modifications for masks). But the ideas we glean: they confirm that ignoring specular points improved matching. They might also mention tuning the detector head to not cluster points around specular boundaries (like high contrast edges at specular highlight edges often attract detectors). Perhaps they used a loss that penalizes detections in masked areas. We might incorporate that in our fine-tuning (e.g., set detector loss =0 for masked pixels so network learns not to detect there).

- **SuperPointTrackingAdaptation (MICCAI 2023)**: This is a more advanced concept – adjusting the feature descriptors online during tracking to improve consistency. The code structure reveals they have integrated COLMAP (supercolmap folder) – likely to run an initial mapping with baseline SuperPoint, and then maybe use tracks from COLMAP to fine-tune descriptor network (maybe using "track consistency loss"). The presence of *pytorch-superpoint* inside suggests they actually fine-tune the same network online. This idea of **tracking adaptation** is powerful: they presumably demonstrate that updating the descriptor on the fly to the current scene yields more reliable matches, extending track lengths and reducing bundle adjustment error. They likely

freeze the detector and only adjust descriptor head weights, or even just a scale on descriptors. It's reminiscent of concept in some literature of local feature "domain adaptation" or "self-calibration" (like SLAMTune, not sure if exists).

Given that was published at MICCAI, they likely reported improvement: e.g., maybe baseline SuperPoint gave ATE X, adapted gave ATE Y (<X). Or number of landmarks increased. They might also mention that they only adapt when enough stable tracks are observed (to avoid drifting descriptors on erroneous matches). If their code is available (though incomplete on readme), we could attempt to replicate some aspects: for instance, after initial mapping of a segment, run a few self-supervised training iterations on keyframes to refine descriptors such that reprojection error of matches in local bundle adjustment is minimized (essentially forcing descriptors of matched points to be more similar, and of non-matches less similar). It's like performing a few steps of contrastive learning on the map points. They likely did something along those lines – maybe using Fua's insight (Pascal Fua is an author, and he has done incremental structure-from-motion refinements).

However, adding that to our pipeline might be beyond initial scope. It's something to note as a future direction: once we have a working pipeline, we could integrate an **online learning module** that takes the map and optimizes the feature representation for that map. Because their result suggests it's beneficial: "Tracking adaptation to improve SuperPoint for 3D recon in endoscopy" implies their adaptation yields fewer track breaks, more points and better 3D.

One risk – online learning might overfit to current scene and not generalize to different area if environment changes drastically (less an issue inside the same colon – environment is similar). They must have had safeguards.

For now, our immediate plan is to use static pre-trained features. But we'll keep this idea in our report's outlook: once baseline is working, incorporating **self-supervised online adaptation** (like Leon's method) could further boost performance, especially in long sequences.

Leon's code limitations: It's AGPL-3.0 (a strong copyleft) [100], meaning if we incorporate it, our code needs to be AGPL too. We might not want that; but we can re-implement the concept independently to avoid licensing issues, or treat it as a separate module that doesn't integrate fully. The SuperPoint code we use is MIT, LightGlue is MIT, so we'd prefer to keep things permissive.

To summarize the **Leon repos takeaway**: They validate our approach choices (masking specularities, potential of online learning). They also provide a data point: they used COLMAP with SuperPoint in endo and apparently got a semi-consistent map (if they can adapt, it means they had a baseline reconstruction). That's encouraging for us to attempt mapping too, not just VO.

Takeaways – Implementation:

- We will use **PyTorch SuperPoint** for training with possibly our augmentation and specularity masking (we might port some logic from Leon's TF code to PyTorch).
- We endorse a **PyTorch-based pipeline** fully, as it eases integration of LightGlue (also PyTorch) and possible future improvements (like plugging in LoFTR which is PyTorch).
- We will maintain reference to original code authors (for academic credit and to ensure we understand any differences).
- LightGlue's code (by Lindenberger and team) is available on GitHub (MIT). We will integrate that – it likely comes as a small library or script. It might require PyTorch 2.0 because they used some compile, but that's fine.

- We have to be mindful of **license compatibility**: MIT and Apache are fine. The only potential trouble is if we considered using code like from ALIKED (which is in an IEEE TIM paper, but their repo Shameless521/ALIKED is MIT, good) or some Kaggle code (which often no license – we'd avoid copying any Kaggle solutions without license).
- The final deliverable code of `colon_matching` presumably will be internal (the user didn't specify open-sourcing, but if they did, we'd double-check license compliance).
- We'll provide a checklist (in next section) detailing steps to set up training and evaluation.

Finally, we proceed with the **Actionable Checklist** for building the benchmark and running baselines.

Actionable Checklist

1. **Assemble Data:** Gather all required datasets. Download **EndoSLAM** (ex-vivo sequences with ground truth) [20], **C3VD** (with depth maps) [21], **EndoMapper** sequences (with calibration and simulated data) [89], and **SCARED** (stereo phantom, via its MICCAI challenge site). Convert all image data to a common format (e.g., undistort EndoMapper images using provided calibration to pinhole camera images, save calibration parameters). Generate specularity masks for each image – e.g., using a simple intensity threshold on the V channel (and maybe also identifying saturated pixels) – store as binary mask images.

2. **Set Up Evaluation Framework:** Implement scripts to compute metrics:

3. For image pairs: if ground truth pose or homography known, compute rotation/translation error and count inliers for mAA [27]. If ground truth correspondences known (stereo), compute precision/recall [83].
4. For sequences: a module to feed frame-by-frame matches into RANSAC odometry estimation (we can use OpenCV's `findEssentialMat` and `recoverPose` for monocular relative pose, or `stereoRectify` for stereo), then accumulate poses and compare to GT (compute ATE, RPE).
5. Use TUM benchmark's tools for ATE/RPE if convenient (they have scripts to align and compute RMSE).
6. Also implement tracking failure detection: if RANSAC finds < some inliers, mark tracking lost and optionally attempt relocalization by matching to older keyframes (optional for extended challenge).

7. Logging of computation time for each module (feature detect, match, RANSAC) to assess speed.

8. **Implement Baseline Methods:** using OpenCV and PyTorch:

9. Integrate OpenCV SIFT, ORB, AKAZE for detection+description. For matching: OpenCV BFMatcher (with crossCheck or ratio test as needed) for SIFT/AKAZE (L2 distance) and ORB (Hamming).
10. Write a wrapper for SuperPoint: either use the pre-trained weights provided by eric-yyjau (COCO model) and run inference using PyTorch (possibly export to ONNX for speed).
11. Integrate SuperGlue and LightGlue: the repos likely come with example code to call given sets of keypoints+descriptors and get matches. Ensure we correctly set the normalization (SuperGlue expects normalized image coords 0–1 possibly).
12. LoFTR: use their inference script (feed two images, get dense matches).

13. For fairness, ensure all methods use the same camera intrinsics when computing epipolar errors (like for RANSAC threshold, we use maybe normalized image coords or a fixed pixel threshold).

14. **Train/Fine-tune SuperPoint on Endoscopy:** Using the PyTorch SuperPoint:

15. Prepare a training set: e.g., take 1000 random frames from EndoMapper (various patients) – ensure masks for specular are applied.
16. Pre-train detector if needed (or use MagicPoint provided by eric-yyjau on synthetic shapes). Possibly we can skip MagicPoint and directly fine-tune descriptors using Homography Adaptation on endo images (since we have an already good detector from COCO).
17. Fine-tune descriptor: generate random homography pairs from the collected endo frames (small perturbations up to, say, 10° rotation and ±5% scale, since larger deformations might not be homography anyway). Use the standard homographic adaptation loss (L2 descriptor loss for corresponding pts) [37] .
18. Also incorporate **mask in training**: when computing detection loss (if we do that), do not penalize absence of points in masked areas and maybe penalize if point is detected in mask (we can add a term that pushes heatmap in mask to zero).
19. Validate during training on a small set (maybe manually create a set of image pairs with known matches via stereo or something to see if matching precision improves).

20. Save the fine-tuned weights; convert them to the same format as inference code expects.

21. **Run Benchmark Baselines:** Execute each baseline on the defined tasks:

22. For each image pair in test set, output matches and feed to evaluation script. Accumulate metrics.
23. For each sequence, run VO (if method is capable) and log trajectory, then compute ATE vs GT.
24. Do not forget to apply specularity mask for those methods that can use it: e.g., for SuperPoint we will zero out detections in mask; for SIFT/ORB, we can also filter out keypoints that fall in masked area post-detection (to be fair, we can do that since those are clearly bad features).

25. Collect the results in a structured way (perhaps a JSON or CSV per method).

26. **Compare and Summarize:** Generate tables/plots:

27. Table of pair matching results: e.g., "HPatches-like" table but for our pairs, listing average precision, recall, mAA, etc. (with citations if referencing any known value).
28. Table of VO results: ATE, RPE, etc.
29. Plot trajectories for qualitative check (include in report).

30. Check key findings: e.g., "Method X had highest recall but at cost of lower precision", "Method Y maintained tracking 90% of frames vs method Z only 50%", etc.

31. **Final Pipeline Selection:** Based on above, finalize that SuperPoint (endotuned) + LightGlue is the best compromise of accuracy and speed for integration into a SLAM. If, hypothetically, LoFTR had significantly better ATE but ran 5× slower, we note that trade-off. Given literature, LoFTR might not dramatically outdo SuperPoint+LightGlue in colon data because LoFTR is better in textureless *rigid* scenes (it leverages context well, but deformation might confound it; plus LightGlue is bridging some gap).

32. Nonetheless, if something like RoMa excelled in our wide-baseline test (maybe identifying loop closures where others failed), we might consider using it as a secondary match method for loop closure detection (since speed isn't an issue if run occasionally).

33. **Integration into `colon_matching` Repo:** Implement the chosen pipeline (SuperPoint+LightGlue) in the existing codebase. The user's repo likely has infrastructure (maybe

it was ORB-SLAM based, adding CUDA SIFT etc.). We'll replace the feature extraction and matching modules with our new ones. Also incorporate the mask usage in real-time (the repo likely already has a function to compute specularity mask since they mentioned adding masks for specularities). Indeed, they said "*ipastore/colon_matching – adds masks to filter keypoints on specularities*." So they have that concept; we will ensure our SuperPoint uses that mask.

34. **Testing in SLAM loop:** Run the full modified SLAM on a couple of sequences (maybe one from EndoMapper). Check if tracking is smooth, how often it relocalizes, etc. If any parameter needs tweaking (like RANSAC threshold maybe needs to be larger due to calibration uncertainties?), adjust and note it.

35. **Documentation & Tables:** Prepare `papers.csv`, `datasets.csv`, `metrics.csv`, `repos.csv` as per request, populating with the data we gathered (citations given in text to ensure credibility, but probably the final CSV can be included in appendix without citations if format requires, but since user asked to preserve citations, we might include them as well).

This checklist ensures we go from raw data to evaluated results and integration of the chosen solution.

Finally, we compile the Appendix tables as required, including definitions in **Glossary** and a short **Risk Register**:

Glossary:

- *mAA (mean Average Accuracy)* – The mean (over multiple thresholds and scenes) of the percentage of image pairs for which the estimated pose is within acceptable error. Specifically, we use thresholds of rotation (0.25°,0.5°,1°,2°,5°,10°) and translation (0.25%,0.5%,1%,2%,5%,10% of scene size or absolute meters depending on dataset) [60], count pair as "accurate" if both errors are below threshold, compute accuracy at each threshold, average over thresholds and scenes [24] [101]. Higher mAA (max 1.0) is better.

- *ATE (Absolute Trajectory Error)* – RMS error between estimated and ground truth trajectory positions after aligning the trajectories (usually with a rigid transform) [25]. Measured in same units as input (mm or cm). Lower is better.

- *RPE (Relative Pose Error)* – Error in the motion between consecutive frames (or frames a fixed delta apart). We compute translational RPE as the norm of difference in translation vectors, and rotational RPE as the angle between rotation quaternions, averaged over the sequence. Often expressed as drift per second or per meter.

- *Inlier Ratio* – After matching and RANSAC, the fraction of matched points that are classified as inliers (i.e., fit the estimated model). 100% means all matches agree with one geometric model (ideal), lower means many outliers. Higher is better.

- *Precision/Recall (of matches)* – Precision = % of reported matches that are correct. Recall = % of true correspondences that were found among matches. These require known ground truth correspondences or transformation to evaluate. We use a tolerance (e.g. 3 px reprojection error) to count a match as correct [83]. High precision means few false matches (robust), high recall means many true points matched (complete). Both important; F1 is their balance.

- *Repeatability* – If a physical point is visible in two images, did the detector find it in both? Computed as #correspondences (within epsilon distance) divided by min(#features in img1, #features in img2) [58]. Usually evaluated for detector alone (no descriptors). High repeatability indicates stable detection under viewpoint/illumination change.

- *SuperPoint* – A self-supervised CNN-based local feature detector & descriptor. Produces semi-dense keypoints (interest points) with 256-d descriptors [34]. Often the baseline for learned features. Abbreviations: SP.

- *LightGlue / SuperGlue* – Learnable matchers that take two sets of keypoints+descriptors and output filtered matches. They use attention (transformers) to reason about match consistency [49]. LightGlue is a faster, efficient version (ICCV 2023) of SuperGlue (CVPR 2020) [49].

- *LoFTR* – Detector-free LOcal Feature TRansformer. Dense matching model that directly predicts correspondences on image grids without discrete keypoints [39]. Good for low-texture cases since it uses global context.

- *RANSAC* – Random Sample Consensus. An iterative algorithm to robustly fit a model (e.g., Essential matrix) by repeatedly sampling minimal sets of matches and selecting the model that has most inliers. We use it to filter wrong matches and compute pose. Parameter: threshold (max allowed reprojection error for a point to be inlier).

- *Specularity Mask* – A binary mask marking pixels that are saturated or near-saturated highlights (specular reflections). These areas violate brightness constancy and usually have no consistent pattern to track once viewpoint changes. By masking, we avoid detecting or using features there.

- *Deformation* – In endoscopy, tissue surfaces move or deform. Non-rigid motion means a single rigid-body transform can't perfectly align points between frames, violating the epipolar geometry assumption. Feature matches on deforming parts either become outliers or require more complex modeling (optical flow or piecewise alignment).

Risk Register:

- **Domain Shift Risk:** Models like SuperPoint trained on natural images might not capture endoscopic textures, risking low feature repeatability. *Mitigation:* Fine-tune on endoscopy data (as we do) and mask out-of-distribution artifacts (specularities) [1]. We will also test on a small held-out set of endo images to ensure features are detecting meaningful points (like blood vessel bifurcations) rather than random noise.

- **Ground Truth Reliability:** For in-vivo data, "ground truth" pose or depth is often approximate (e.g., CT alignment might have errors due to deformation). If our benchmark takes those as truth, we might penalize a method for "errors" that are actually not its fault but due to anatomy move. *Mitigation:* Use ex-vivo or simulated data with known geometry for primary metrics. For in-vivo, focus on relative metrics or manual verification (like loop closures correctness by visual inspection). Also incorporate the EndoMapper SfM pseudo-truth carefully: treat it as *approximate* truth, perhaps with a tolerance (like consider a pose "correct" if within 2–3 cm since the model itself might have that error).

- **Non-Rigidity Impact:** The assumption of rigid structure-from-motion breaks in parts of our sequences, which can confuse algorithms and evaluation metrics. E.g., an algorithm might get a

low mAA because the scene deformed, not because it matched wrong. *Mitigation:* In metrics like mAA, use relatively loose thresholds for in-vivo sequences to avoid labeling slight deformations as failure. Or evaluate in segments where deformation is minimal (pause when too much motion). Possibly integrate a deformable mapping approach in future, but that's beyond current scope.

- **Overfitting to Benchmark:** If we train SuperPoint heavily on our specific dataset, it might overfit (e.g., learning to detect particular endoscope imaging artifacts that don't generalize). *Mitigation:* Use varied data (multiple patients, plus augmentations). Monitor performance on separate test scenarios (like use EndoSLAM pig for training, test on C3VD human, etc.) to ensure generalization across slight domain shifts (pig vs human tissue differs in appearance).

- **Real-time constraints:** The ultimate goal is a SLAM that runs live. Using heavy matchers might slow it down. *Mitigation:* We choose LightGlue for efficiency and can further reduce feature count or use GPU acceleration. We will measure runtime. If our pipeline is borderline for 30fps, consider pruning (e.g., skip matching on every frame, perhaps match every other frame or use temporal propagation of points).

- **Integration issues:** Replacing ORB features with our learned features in an existing ORB-SLAM-derived system might break some assumptions (like ORB-SLAM's covisibility graph parameters tuned for ~500 ORB points per frame). *Mitigation:* We will adjust those system parameters (for example, number of keypoints, scale pyramid) to fit SuperPoint. Also ensure keyframe decision criteria (which often depend on number of features tracked) are updated to reflect our feature counts. We'll test the integrated system on a short sequence to verify it still performs mapping and loop closure as expected.

By following this approach, we address the user's five special questions explicitly and provide a blueprint and justification for each component of the colonoscopy SLAM pipeline. The resulting system will leverage **SuperPoint (domain-tuned) + LightGlue** as the core, validated by our thorough benchmark that covers both classical and modern methods, and informed by the current state-of-the-art insights gathered above.

(The following tables and figures provide supporting details and structured data as requested.)

Appendices

papers.csv (each row: a paper/method with key attributes):

Title / Method	Venue & Year	Task	Domain	Method Category	Key Idea / Contribution	Datasets Used
SIFT – Scale Invariant Feature Transform (Lowe)	ICCV 2004 (award)	Local feature detection & descriptor	General (natural images)	Sparse, handcrafted	DoG pyramid for scale-invariant keypoints; 128-d gradient histograms descriptor [34]	Oxford affine dataset; Graf, Wall, etc.
ORB – Oriented FAST and Rotated BRIEF	ICCV 2011	Local feature detection & descriptor	General (wide baseline, real-time)	Sparse, handcrafted (binary)	FAST corner detector + Harris score; BRIEF descriptor with orientation [102]	Author's dataset (indoor)
SuperPoint (DeTone et al.)	CVPR 2018 (WACV '18 poster)	Local feature detector & descriptor	General (COCO, Synthetic shapes)	Sparse, learned CNN	Self-supervised joint detection & description; train on synthetic homographies [34]	MS-COCO + HPatches for eval
SuperGlue (Sarlin et al.)	CVPR 2020	Feature matching (graph NN)	General (outdoor/ indoor)	Sparse, learned matcher	GNN with self & cross-attention to match SuperPoint features; uses sinkhorn matching [38]	PhotoTourism (MegaDepth), ScanNet (indoor)
LightGlue (Lindenberger et al.)	ICCV 2023	Feature matching (efficient)	General (COCO, MegaDepth)	Sparse, learned matcher	Refines SuperGlue by adaptive pruning of keypoints and efficient attention windows [74] [48]	COCO + MegaDepth (synthetic pose pairs)

Title / Method	Venue & Year	Task	Domain	Method Category	Key Idea / Contribution	Datasets Used
LoFTR (Sun et al.)	CVPR 2021	Detector-free correspondence	General (outdoor, indoor)	Dense, Transformer	Matches all-to-all with coarse-to-fine transformers; no keypoints needed [39]	MegaDepth (outdoor), ScanNet (indoor)
RoMa (Edstedt et al.)	CVPR 2024	Detector-free dense matching	General (broad)	Dense, Hybrid (ViT+CNN)	Uses frozen DINOv2 ViT for coarse features + learned CNN for fine; robust sampling strategy [40]	MegaDepth, & synthetic edge images
ALIKED (Zhao et al.)	IEEE TIM 2023	Local features (det+descr)	General (day-night)	Sparse, learned (deformable conv)	Lighter-weight SuperPoint variant with deformable conv for viewpoint robustness [44] [43]	Aachen Day-Night + COCO (for training)
XFeat (Potje et al.)	CVPR 2024	Local features (fast)	General (any)	Sparse, learning-based	MobileNet backbone + dynamic pooling to extract features fast; optimizes inference speed [105]	COCO (Synthetic homography training)
(Medical) SuperPoint-Endo (León et al.)	MIDL 2022 (LNCS 13754)	Local features in colonoscopy	Endoscopy (colonoscopy)	Sparse, learned (adaptation)	SuperPoint fine-tuned on endoscopy; avoids keypoints on specular highlights [1]	Private colonoscopy dataset (Real, ~20 videos)

Title / Method	Venue & Year	Task	Domain	Method Category	Key Idea / Contribution	Datasets Used
(Medical) SuperPoint Tracking Adaptation (León et al.)	MICCAI 2023 (LNCS 13970)	SLAM feature adaptation	Endoscopy (colonoscopy)	Sparse, learned on-the-fly	Online fine-tuning of SuperPoint descriptors during SLAM using track consistency [106] [107]	EndoMapper + Phantom (for eval)

(Table: Key research papers and methods with their attributes. "General" domain means broad scenes; medical-specific works at bottom.)

datasets.csv (key public datasets for endoscopy SLAM/matching):

Dataset & Year	Modality	Organ / Procedure	# Sequences (Frames)	Resolution & FPS	Ground Truth Provided	Calibration
EndoSLAM (2021) [19] [20]	Monocular (RGB); also Capsule	**Ex-vivo pig GI tract** (colon & stomach); plus synthetic and capsule parts	**35 sequences** ex-vivo (18 colon, 12 stomach, 5 others), each ~300–1000 frames; Synthetic: 3 colon sims	256×256 (ex-vivo video frames) @ 30 Hz; synthetic 768×768	**6-DoF Pose** for ex-vivo (optical/EM tracking) [20]; depth for synthetic; no GT for capsule	Yes (intrinsics & distortion for scope)
SCARED (MICCAI 2019)	Stereo RGB	**Phantom abdomen** (with liver, etc.) – **laparoscopy**; + one in-vivo sequence	4 phantom seq (rigid), 1 in-vivo (partial deform), ~300 frames each	1024×1024 @ 30 Hz (stereo)	**Camera poses** (from optical tracking) for all frames; **Stereo ground-truth disparity** on textured surfaces (calibration object)	Yes (stereo intrinsics, extrinsics)

Dataset & Year	Modality	Organ / Procedure	# Sequences (Frames)	Resolution & FPS	Ground Truth Provided	Calibration
C3VD – Colonoscopy 3D Video Dataset (2022) [85] [21]	Monocular RGB (with CT reference)	**In-vivo human colon** (screening colonoscopies)	**22 sequences** (short segments), total **10,015 frames**	854×480 @ ~25–30 Hz (after processing)	**Depth map per frame** (from CT-Video registration, ~mm accuracy) [21] ; **Surface normals, optical flow, occlusions**; and estimated **6-DoF pose** in CT coordinate [21]	Yes (intrinsic calibrated, distortion removed)
EndoMapper (Nat. Sci. Data 2023) [87] [88]	Monocular HD RGB (endoscope)	**In-vivo human GI** (96 full procedures: colonoscopies & gastroscopies)	**96 sequences** (72 colon, 24 gastro); ~24h video total (~2.59 million frames)	~1920×1080 (colon), 1920×1440 (gastro) @ 25–30 Hz	**No ground truth poses** for real; but **3 simulated colon sequences** with known deforming mesh & camera poses [110] ; plus **SfM reconstructions** (sparse 3D + up-to-scale poses) for ~30 segments [25]	Yes (intrinsic calibrations for each scope, and calibration checkerboard videos) [87]
Hamlyn Endoscopic Video (2017)	Stereo & Mono RGB	**Surgical endoscopy** (various: prostate, ENT sinus, etc.)	~10 stereo seq, ~>30 mono seq (short clips)	640×480 to 1280×1024, 30 Hz	Some have **hand–eye tracked poses** (for robot endoscope) and **CAD models** for reference; others just video + annotations	Partial (for stereo, yes; mono calibration partly provided)

Dataset & Year	Modality	Organ / Procedure	# Sequences (Frames)	Resolution & FPS	Ground Truth Provided	Calibration
Heidelberg Colon Laparoscopy (2019) [92]	Stereo RGB (laparoscope)	**Ex-vivo pig colon** (external view via laparoscopy)	1 dataset (images from lap surgeries)	1920×1080 stereo	**3D point clouds** of colon surface (from stereo reconstruction)	Yes (stereo calib)
Colon 10k (2020) [113]	Monocular frames	**In-vivo colon** (polyp patches)	10,000 images (from short clips)	854×480	No poses (used for place recognition)	Yes

(Table: Public datasets relevant to GI endoscopy SLAM/matching. Emphasis on provided ground truth and calibration.)

metrics.csv (each row a metric with definition and note):

Metric	Definition / Formula	Typical Thresholds / Criteria	Strengths for Endo	Limitations for Endo (if any)
Repeatability (of detector)	% of true scene points detected in both images. If keypoint p_i in image1 lies within ε pixels of a keypoint in image2 (after projecting via known transform), it's a repeat. $ \text{Repeatability} = \frac{#\text{repeat detections}}{\min(n_1, n_2)} $ [58] .	ε = 3 px or 5 px (commonly); evaluated on overlapping image regions.	Focuses on detector robustness to viewpoint. In endo, highlights if features survive specular/blur changes.	Hard to compute without known geometric transform; deformations violate assumption (point might move). We use on stereo or consecutive frames only.

Metric	Definition / Formula	Typical Thresholds / Criteria	Strengths for Endo	Limitations for Endo (if any)
Localization Error	The distance (in pixels) between a detected keypoint and the true projected location of that scene point. If ground truth correspondence is (x_{gt}, y_{gt}), and detector gives (x, y), error = $\sqrt{(x-x_{gt})^2+(y-y_{gt})^2}$. Report mean or distribution.	Evaluate for repeat points; sometimes threshold at 1 px to count "accurate".	Ensures keypoint positions are precise (important for triangulation accuracy in SLAM). Endo: ensures e.g. a vessel bifurcation is marked exactly, not off by 5px.	Requires ground truth points or very reliable matches (maybe from high-res stereo). In endo, slight tissue motion can make "ground truth" uncertain, so use with caution.
Matching Precision	Among predicted matches, the fraction that are correct. $\text{Prec} = \frac{\#\text{correct matches}}{\#\text{total matches}}$. A match is "correct" if it associates the same 3D point (or within tolerance of ground truth alignment) [83].	Tolerance: often 3 px reprojection error or 5 px in 1080p images; epipolar error < 1 px if pose known.	High precision means minimal outliers – critical for reliable pose estimation (RANSAC might succeed more). For endo, precision ensures we're not matching unrelated patterns (which can happen in self-similar texture).	Doesn't account for missed matches (that's recall). A method that matches very few points might achieve 100% precision but be useless (so must consider recall too). Also needs GT correspondence.

Metric	Definition / Formula	Typical Thresholds / Criteria	Strengths for Endo	Limitations for Endo (if any)
Matching Recall	Among all true correspondences (points that should match), the fraction that the algorithm matched. $ \text{Recall} = \frac{#\text{correct matches}}{#\text{all possible matches}} $. "All possible" can be defined via overlap of detected points with ground truth points visible in both images [83].	If using known homography or depth, "true correspondences" = points in common overlap region. Perfect recall = 1.	Indicates how complete the matching is – important for mapping completeness (more matches = more points triangulated). In colon, high recall means even low-contrast areas got matches, aiding dense mapping.	High recall methods might introduce more false matches (there's a trade-off with precision). Also computing denominator is tricky if detection differs; often approximate by repeatability * min(#feat1, #feat2).
Matching F1-score	Harmonic mean of precision and recall: $\text{F1} = 2 \frac{Prec \cdot Rec}{Prec + Rec}$. Reaches 1 only if both prec and rec are 1.	We may report F1 at a fixed tolerance (like 3px).	Single metric summarizing overall matching quality. Useful to rank methods that have different balance. In endo, F1 rewards methods that get many correct matches without too many outliers – ideal for VO.	Can hide whether a method biases towards precision or recall. Also heavily influenced by how "all possible matches" is defined (if repeatability is low, recall suffers even if matching stage is perfect).

Metric	Definition / Formula	Typical Thresholds / Criteria	Strengths for Endo	Limitations for Endo (if any)
Homography Accuracy (HA)	Given a known homography H_{true} and an estimated H, measure image alignment. Typically reported as AUC of the fraction of correspondences with reprojection error < thresholds. e.g. HA@3px = % of correspondences aligned <3px [37]. Or an integrated AUC for 1–10px.	1px, 3px, 5px thresholds common (for 600×800 images). Also area under curve up to 10px error [37].	Good for planar scenes or to evaluate descriptor matching without doing epipolar geometry. Was base metric for HPatches. Endo use: perhaps for evaluating local patch alignment on tissue surfaces which are locally planar.	Not directly applicable if scene not planar or if no homography ground truth. In endo, rarely a pure homography (except perhaps in a flat phantom). So this metric is secondary – we may use in specific scenarios (like evaluating on a printed checkerboard phantom).

Metric	Definition / Formula	Typical Thresholds / Criteria	Strengths for Endo	Limitations for Endo (if any)
Essential Matrix / Pose Error (mAA)	After estimating pose (R, t) between images, measure error vs ground truth. Typically, rotation error = angle between estimated and GT rotation; translation error = angle between estimated and GT translation directions (for monocular, or Euclidean distance if scale known). **Average Accuracy (AA)** = fraction of pairs with errors below threshold (e.g. <5° & <5cm) [24]. **mAA** = mean of AA over multiple thresholds [114].	IMC uses thresholds: Rot = 0.25°,0.5°,1°,2°,5°, 10°; Trans = 0.25%,0.5%,1%, 2%,5%,10% of scene (or fixed distances) [60]. We can similarly use degree and cm thresholds. mAA is area under the curve of accuracy vs threshold, basically.	Directly reflects end goal: camera pose accuracy. mAA is a single number that penalizes both large errors and registration failures [24] [9]. In endo, if a method fails to match enough to estimate pose, mAA goes down (which is desired, as that method isn't useful).	mAA conflates failure to register with actual pose precision [9]. If many pairs are unregistered, mAA is very low even if ones that did register were accurate. We mitigate by also reporting % of registered frames (N_img) [65]. Also, because colon can deform, an estimated pose might be "wrong" because scene moved – so mAA can be harsh on methods in deformable segments. We may restrict to near-rigid segments for mAA evaluation.

Metric	Definition / Formula	Typical Thresholds / Criteria	Strengths for Endo	Limitations for Endo (if any)
Inlier Ratio (post-RANSAC)	$ \frac{#\text{inliers}}{#\text{total matches}}$ after running geometric verification (like RANSAC for Essential matrix). Inliers are matches consistent with estimated model within a threshold.	No fixed threshold – inherent to RANSAC (we use e.g. 1px reprojection for inlier definition). Often average inlier % reported.	Indicates how well the matcher + model fitting cleaned out outliers. High inlier ratio means most matches are geometrically consistent (good for bundle adjustment). In endo, it can reveal if many features come from moving tissue (then they won't fit single model, lowering ratio).	A method might intentionally throw away many matches to keep only confident ones – yielding high inlier ratio but maybe few inliers. So interpret alongside total inliers count. In endo, non-rigid motion will inherently limit the achievable inlier ratio even for an optimal method (since one rigid model can't explain all matches).
Absolute Trajectory Error (ATE)	For a sequence of camera poses ${T_i} $ (estimated) and $ {T_i^{gt}}$ (ground truth), ATE = RMS of position errors after aligning the trajectories. Typically we find a rigid transform S that best aligns estimated to GT (to account for arbitrary start pose or scale) and compute $\text{ATE} = \sqrt{\frac{1}{N} \sum_i \| S(T_i) - T_i^{gt} \|^2 }$ [25].	– (units of distance, e.g. mm or cm). Sometimes median or mean reported instead of RMS.	Gold standard for evaluating SLAM trajectory accuracy globally. In endoscopy, if we have GT (e.g. EM tracker or CT overlay), ATE shows overall drift. Lower ATE means map and trajectory are globally consistent.	Requires ground truth trajectory (difficult in vivo). Also, if colon deforms, even a perfect VO will incur "ATE" because environment shifted – so ATE might unfairly penalize in those cases. For ex-vivo or rigid phantom, it's great. We'll mostly use on sequences with external tracking or simulation.

Metric	Definition / Formula	Typical Thresholds / Criteria	Strengths for Endo	Limitations for Endo (if any)
Relative Pose Error (RPE)	Measures drift per step. For a given time interval Δ (frames apart), compare the motion: $\Delta T_{i} = T_{i}^{-1} T_{i+\Delta}$ (estimated) vs ΔT_{i}^{gt}. Compute error transformation $E_i = (\Delta T_i)^{-1} \Delta T_i^{gt}$, then report translation error = ‖trans(E_i)‖ and rotation error = angle(E_i). Average over i (often RMS).	Common: Δ = 1 frame (consecutive) for visual odometry; or Δ = some fixed time (e.g. 1s) for scale drift evaluation. Reported as e.g. "0.5% translational drift per meter" by normalizing RPE over distance traveled.	Focuses on local accuracy – important for odometry smoothness (no large frame-to-frame jumps). In endo, a robust tracker should have low RPE for short gaps (meaning it's consistent locally). Also helps diagnose if drift is linear or compounding.	Not sensitive to global drift (doesn't reflect loop closure success). Also, in very low-texture parts, VO might do okay frame-to-frame (low RPE) but accumulate error (which ATE catches but RPE doesn't fully). Use both. For colon, RPE might remain low even if colon is deforming (the system might track the deformation as if motion).
Track Length Distribution	In a video, build feature tracks: how many frames does each feature remain matched/tracked? We can report distribution (e.g. % of tracks lasting ≥5 frames, ≥10 frames, etc.) or average track length.	– (we often give median track length, and maybe 90th percentile). ORB-SLAM's median track length is ~length of temporal window used (since it discards after certain time).	Longer tracks indicate more stable features and better inter-frame matching. In endoscopy, longer tracks mean the system can do triangulation from multiple views, improving map accuracy. Good for evaluating effect of, say, descriptor adaptation (expected to increase track length).	Doesn't directly measure geometric accuracy, just consistency. Could be inflated if the method falsely keeps tracking a feature that changed (e.g., mistaking one specular blob for another across frames – track length high but incorrect). We can mitigate by checking if long tracks are geometrically plausible.

Metric	Definition / Formula	Typical Thresholds / Criteria	Strengths for Endo	Limitations for Endo (if any)
Loop Closure Recall / Precision	When the camera revisits a previously seen area, does the system recognize it? **Recall** = (# of actual loop occurrences detected) / (# of actual loops). **Precision** = (# of correct loop detections) / (# of loop detections made). A “loop detection” is typically counted if the system’s place recognition matched frames that are truly same place (within some tolerance in space).	For colonoscopy, define “loop” loosely: could be the same segment seen on insertion vs withdrawal. We might manually label a few such correspondences as ground truth. Aim for high recall without false positives.	Measures global consistency ability. Important if we want map joining of forward/ backward passes. High recall means algorithm rarely misses a revisited spot, aiding relocalization in SLAM. Precision important to avoid incorrect loop closures that can severely corrupt the map.	Defining ground truth loops is tricky in deformable environment. Also, an algorithm might avoid declaring loop closures if uncertain (thus high precision but low recall). In colon, loops are not obvious except maybe at known anatomical landmarks. We might approximate by region labels (EndoMapper’s region annotations: if it matches frames both labeled “cecum”, that’s sort of a loop closure).

Metric	Definition / Formula	Typical Thresholds / Criteria	Strengths for Endo	Limitations for Endo (if any)
Specularity-Masked Metrics	Any of the above metrics computed after removing or ignoring specular highlight regions. E.g., masked repeatability = repeatability considering only keypoints outside specular mask; masked precision = count a match as wrong if it involves a masked point, etc.	– (just apply mask before evaluation).	Specifically evaluates performance on *valid tissue features* ignoring the distracting specular points. If a method naively detects specular points, its masked repeatability will be low (since those points don't repeat) – a masked metric will penalize that correctly. It basically focuses evaluation on meaningful regions.	If a method smartly ignores speculars, its masked vs unmasked metrics might be similar (which is good). We must ensure we generate the masks consistently (slight risk of misidentifying some specular vs actual feature as mask). Also, masked precision might slightly inflate scores (because we removed all obviously bad areas). But for fairness between methods, it's fine.

(Table: Definitions and discussion of metrics relevant to our benchmark.)

repos.csv (comparing relevant code repositories):

Repo (Link)	Language/ Framework	Purpose (Training/ Inference)	Maintenance & Activity	License	Notable Features and Tools
eric-yyjau/pytorch-superpoint [33]	Python (PyTorch)	Training & Inference for SuperPoint (end-to-end)	Active (last commit mid-2023; 100+ stars)	MIT	Supports MagicPoint pre-training and SuperPoint joint training [33]; COCO and KITTI configs included; TensorBoard logging; easy inference (exports ONNX).
rpautrat/SuperPoint (original TF)	Python (TensorFlow 1.x)	Training code for SuperPoint (MagicPoint and SP)	Static (no updates since 2019)	Apache 2.0	Implements homographic adaptation training exactly as paper; provided pre-trained weights for MagicPoint on synthetic and SP on COCO.
magicleap/SuperGlue Pretrained Network	Python (PyTorch)	Inference for SuperGlue (with pre-trained weights)	Static (one-time code release in 2020)	MIT	Easy integration: just feed keypoints+descriptors and get matches; includes demo with SuperPoint.

Repo (Link)	Language/ Framework	Purpose (Training/ Inference)	Maintenance & Activity	License	Notable Features and Tools
LightGlue (by P. Lindenberger et al.)	Python (PyTorch Lightning)	Inference (with code to train on custom data if needed)	Active (2023 initial release; minor fixes after)	MIT	Provides pre-trained weights for indoor/ outdoor. Written in Lightning for easy deployment; supports batch matching (multiple image pairs). Very efficient memory usage.
Parskatt/LoFTR (official)	Python (PyTorch)	Inference (training code in original repo under configs)	Static (after CVPR'21, mainly maintenance)	MIT	Easy API: give two images, returns dense matches + confidence. CUDA optimized correlation. Training scripts for MegaDepth provided (needs large GPU).
JohanEdstedt/RoMa (CVPR 2024)	Python (PyTorch)	Inference (matching library)	New (2024 release, being adopted)	MIT	Code includes pre-trained model and example on matching image pairs. Utilizes DINOv2 backbone (must download 1GB model) [50] . Has option for different match refinement settings (coarse only or coarse+fine).

Repo (Link)	Language/ Framework	Purpose (Training/ Inference)	Maintenance & Activity	License	Notable Features and Tools
Shameless521/ALIKED	C++/Python (PyTorch)	Inference (with demo) – also training code in PyTorch	Fairly active (last update late-2022)	MIT	C++ demo for real-time feature extraction; Python training code to reproduce paper (requires TIM dataset). Has deformable conv via CUDA kernels (needs setup).
VerLab/XFeat (accelerated features)	Python (PyTorch)	Training & inference for XFeat (CVPR'24)	New (2024), perhaps not widely adopted yet	MPL-2.0 (Copyleft)	Training pipeline for speed-optimized network on homography tasks. Inference yields 64-d descriptors quickly. Provided timing scripts.

Repo (Link)	Language/ Framework	Purpose (Training/ Inference)	Maintenance & Activity	License	Notable Features and Tools
UNIZAR: LeonBP/ SuperPointEndoscopy	Python (TF1 & OpenCV)	Training (with endoscopy data) & inference demo	Sparse updates (2022 last commit)	AGPL-3.0	Code to generate specularity masks from images and exclude them in training; custom homography augmentation (maybe “ga” stands for geometry-aware?). Trains on MIDL dataset.
UNIZAR: LeonBP/ SuperPointTrackingAdaptation	Python (PyTorch + COLMAP)	Adaptation module on top of SuperPoint (requires running COLMAP BA)	Research code (2023, not polished for general use)	AGPL-3.0	Wraps COLMAP for bundle adjustment, then fine-tunes SP descriptors by minimizing reprojection errors (likely uses BA residuals as supervision). Automates iterative process: track -> BA -> adapt -> repeat.

(Table: Comparison of code repositories for key components. Emphasis on which we choose and why.)

---

1 34 58 94 [2203.04302] SuperPoint features in endoscopy
https://arxiv.org/abs/2203.04302

2 3 4 5 6 7 8 9 10 11 12 23 24 26 27 28 29 30 31 36 37 38 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 95 96 98 99 101 103 104 105 114 (PDF) Mismatched: Evaluating the Limits of Image Matching Approaches and Benchmarks
https://www.researchgate.net/publication/383529999_Mismatched_Evaluating_the_Limits_of_Image_Matching_Approaches_and_Benchmarks

13 14 15 22 25 87 88 89 90 92 97 110 111 112 113 Endomapper dataset of complete calibrated endoscopy procedures | Scientific Data
https://www.nature.com/articles/s41597-023-02564-7?error=cookies_not_supported&code=b91c0758-5832-4757-a238-a69bec13f325

16 17 18 83 93 Feature matching for texture-less endoscopy images via superpixel vector field consistency - PMC
https://pmc.ncbi.nlm.nih.gov/articles/PMC9045917/

19 EndoSLAM dataset and an unsupervised monocular visual ...
https://www.sciencedirect.com/science/article/abs/pii/S1361841521001043

20 108 EndoSLAM Dataset - Mendeley Data
https://data.mendeley.com/datasets/cd2rtzm23r/1

21 85 Colonoscopy 3D video dataset with paired depth from 2D-3D ...
https://pmc.ncbi.nlm.nih.gov/articles/PMC10591895/

32 AdaLAM: Revisiting Handcrafted Outlier Detection | Request PDF
https://www.researchgate.net/publication/342027472_AdaLAM_Revisiting_Handcrafted_Outlier_Detection

33 eric-yyjau/pytorch-superpoint: Superpoint Implemented in ... - GitHub
https://github.com/eric-yyjau/pytorch-superpoint

35 [PDF] arXiv:2104.00680v1 [cs.CV] 1 Apr 2021
https://arxiv.org/pdf/2104.00680

39 [PDF] DKM: Dense Kernelized Feature Matching for Geometry Estimation
https://openaccess.thecvf.com/content/CVPR2023/papers/Edstedt_DKM_Dense_Kernelized_Feature_Matching_for_Geometry_Estimation_CVPR_2023_paper.pdf

84 EndoGSLAM: Real-Time Dense Reconstruction and Tracking ... - arXiv
https://arxiv.org/html/2403.15124v1

86 109 "Ground truth" colonoscopy dataset to boost cancer detection ...
https://www.bme.jhu.edu/news-events/news/researchers-release-dataset-to-evaluate-computer-vision-in-colonoscopy/

91 O. León Barbed - CV
https://sites.google.com/unizar.es/leon-barbed/inicio

100 GitHub - LeonBP/SuperPointTrackingAdaptation: Tracking adaptation to improve SuperPoint for 3D reconstruction in endoscopy
https://github.com/LeonBP/SuperPointTrackingAdaptation

102 GitHub - ericzzj1989/Awesome-Image-Matching: Bibliographic list for papers of image matching
https://github.com/ericzzj1989/Awesome-Image-Matching

106 Tracking Adaptation to Improve SuperPoint for 3D Reconstruction in ...
https://dl.acm.org/doi/10.1007/978-3-031-43907-0_56

107 Tracking Adaptation to Improve SuperPoint for 3D Reconstruction in ...
https://www.springerprofessional.de/en/tracking-adaptation-to-improve-superpoint-for-3d-reconstruction-/26112110